TỔNG CÔNG TY BƯU ĐIỆN VIỆT NAM

TRUNG TÂM ĐỔI MỚI SÁNG TẠO – PHÒNG AI

Triển khai mô hình AI trong lĩnh vực Chatbot/Trợ lý ảo, ứng dụng các kỹ thuật RAG hoặc fine-tuning mô hình ngôn ngữ lớn có khả năng trả lời thông minh, chính xác, ngữ cảnh hóa câu trả lời dựa trên tài liệu, dữ liệu nội bộ hoặc thông tin chuyên ngành của Tổng công ty.

BÁO CÁO KHOA HỌC

Kính gửi: Ban lãnh đạo Tổng công ty Bưu điện Việt Nam

Đơn vị thực hiện: Phòng AI, Trung tâm Đổi mới Sáng tạo

Ngày báo cáo: 13/10/2026

2026

Lời nói đầu

Trong bối cảnh chuyển đổi số của ngành bưu chính, việc ứng dụng trí tuệ nhân tạo vào công tác tra cứu, hỏi đáp và hỗ trợ nghiệp vụ là một hướng đi có tính thực tiễn cao. Báo cáo này trình bày kết quả nghiên cứu và đề xuất triển khai một trợ lý ảo thông minh dựa trên mô hình ngôn ngữ lớn (LLM) kết hợp kỹ thuật Retrieval-Augmented Generation (RAG) và fine-tuning, phục vụ nhu cầu trả lời chính xác, ngữ cảnh hóa dựa trên tài liệu nội bộ của Tổng công ty Bưu điện Việt Nam (Vietnam Post).

Báo cáo được xây dựng trên cơ sở khảo sát hệ thống và dữ liệu thực tế đang vận hành, kết hợp phân tích lý thuyết và kết quả thực nghiệm. Các nội dung chính gồm: cơ sở lý thuyết về RAG và fine-tuning; kiến trúc hệ thống đa tác tử (multi-agent) đề xuất; kết quả triển khai thực nghiệm; đánh giá định lượng và định tính; cùng lộ trình và khuyến nghị triển khai.

Phòng AI xin gửi lời cảm ơn tới Ban lãnh đạo và các đơn vị liên quan đã tạo điều kiện, chia sẻ bối cảnh nghiệp vụ và dữ liệu để hoàn thành báo cáo này.

Phòng AI, Trung tâm Đổi mới Sáng tạo

Tóm tắt điều hành

Trong bối cảnh chuyển đổi số của ngành bưu chính, Tổng công ty Bưu điện Việt Nam (Vietnam Post) sở hữu một khối lượng lớn tài liệu nghiệp vụ nội bộ: quy trình, quyết định, biểu phí dịch vụ, quy định về hàng hóa và chính sách bồi thường, cùng các nghiệp vụ tra cứu bưu gửi và bưu cục. Việc tra cứu thủ công tốn nhiều thời gian, trong khi các kênh hỏi đáp truyền thống khó đáp ứng nhu cầu tra cứu tức thời, chính xác và có trích nguồn. Báo cáo này nghiên cứu việc triển khai một trợ lý ảo thông minh dựa trên mô hình ngôn ngữ lớn (LLM) kết hợp kỹ thuật Retrieval-Augmented Generation (RAG) và fine-tuning, hướng tới câu trả lời chính xác, ngữ cảnh hóa và bám sát tài liệu nội bộ.

Về phương pháp, báo cáo khảo sát cơ sở lý thuyết về LLM, RAG và fine-tuning; thiết kế kiến trúc trợ lý ảo đa tác tử (multi-agent) gồm một agent điều phối và các agent chuyên biệt, giao tiếp qua các giao thức MCP và A2A; triển khai pipeline RAG lai (hybrid) kết hợp sparse retrieval BM25 và dense retrieval trên vector database Qdrant, kèm reranker; đồng thời đề xuất pipeline fine-tuning cho cả ba loại mô hình: LLM (LoRA/QLoRA), embedding model và reranker, trong đó thí điểm huấn luyện và đánh giá thật embedding model. Toàn bộ mô hình được phục vụ cục bộ (on-premise) bằng vLLM để bảo đảm dữ liệu nội bộ không rời khỏi hạ tầng của Vietnam Post.

Kết quả triển khai cho thấy dense retrieval mạnh ở việc bắt ngữ nghĩa khi câu hỏi diễn đạt khác với từ khóa trong tài liệu, trong khi BM25 bổ trợ tốt cho từ khóa chính xác và số hiệu văn bản; hybrid retrieval đảm bảo phạm vi ứng viên đầy đủ. Phân tách lỗi giữa khâu retrieval và khâu rerank cho phép định vị rõ điểm cần cải thiện, và quan sát thực tế cho thấy khâu rerank đang là điểm nghẽn cần xử lý trước. Ở phần fine-tuning, embedding model đã được thí điểm huấn luyện theo học tương phản và đạt mức cải thiện tương đối khoảng 29–33% trên các chỉ số Accuracy@1, Accuracy@5 và NDCG@10 so với mô hình nền. Báo cáo cũng trình bày kết quả hiệu năng của vLLM, hiệu quả vận hành theo hướng định tính, và đề xuất kiến trúc hybrid RAG + fine-tuning cùng lộ trình phát triển. Kết quả cho thấy RAG là giải pháp phù hợp và khả thi cho tri thức động, còn fine-tuning bổ trợ cho phong cách và nghiệp vụ ổn định, và sự kết hợp của hai hướng tiếp cận mang lại giá trị

thực tiễn cao cho Tổng công ty Bưu điện Việt Nam.

Từ khóa: trợ lý ảo, chatbot, LLM, RAG, fine-tuning, LoRA, hybrid retrieval, reranker, multi-agent, Vietnam Post.

Danh mục từ viết tắt

Từ viết tắt	Diễn giải
A2A	Agent-to-Agent protocol – giao thức giao tiếp giữa các agent
API	Application Programming Interface – giao diện lập trình ứng dụng
BM25	Best Matching 25 – hàm xếp hạng sparse retrieval dựa trên xác suất
CoT	Chain-of-Thought – chuỗi suy luận trong prompt
CSKH	Chăm sóc khách hàng
DPO	Direct Preference Optimization – tối ưu hóa theo sở thích trực tiếp
DPR	Dense Passage Retrieval – dense retrieval đoạn văn
EMS	Express Mail Service – dịch vụ chuyển phát nhanh
FP8	Định dạng số thực dấu phẩy động 8 bit
GPU	Graphics Processing Unit – đơn vị xử lý đồ họa
K8s	Kubernetes – nền tảng điều phối container
LLM	Large Language Model – mô hình ngôn ngữ lớn
LoRA	Low-Rank Adaptation – thích ứng hạng thấp
MCP	Model Context Protocol – giao thức ngữ cảnh mô hình
MLflow	Nền tảng quản lý vòng đời học máy
MRR	Mean Reciprocal Rank – nghịch đảo thứ hạng trung bình
nDCG	normalized Discounted Cumulative Gain
OCR	Optical Character Recognition – nhận dạng ký tự quang học
OTel	OpenTelemetry – chuẩn quan sát hệ thống
PEFT	Parameter-Efficient Fine-Tuning – fine-tuning hiệu quả tham số
PII	Personally Identifiable Information – thông tin định danh cá nhân

Từ viết tắt	Diễn giải
QLoRA	Quantized LoRA – LoRA trên mô hình lượng tử hóa (quantization)
Qdrant	Vector database mã nguồn mở
RAG	Retrieval-Augmented Generation – sinh tăng cường truy hồi
RBAC	Role-Based Access Control – kiểm soát truy cập theo vai trò
RLHF	Reinforcement Learning from Human Feedback
SFT	Supervised Fine-Tuning – fine-tuning có giám sát
SSO	Single Sign-On – đăng nhập một lần
Vietnam Post	Tổng công ty Bưu điện Việt Nam
vLLM	Thư viện phục vụ suy luận LLM hiệu năng cao

Mục lục

Lời nói đầu **1**

Tóm tắt điều hành **2**

Danh mục từ viết tắt **4**

1 Giới thiệu **12**
1.1 Bối cảnh và lý do chọn đề tài 12
1.2 Mục tiêu nghiên cứu 13
1.3 Đối tượng và phạm vi nghiên cứu 14
1.3.1 Đối tượng nghiên cứu 14
1.3.2 Phạm vi nghiên cứu 14
1.4 Phương pháp nghiên cứu 14
1.5 Đóng góp của đề tài 15
1.6 Cấu trúc báo cáo 16

2 Cơ sở lý thuyết và tổng quan **17**
2.1 Chatbot và trợ lý ảo 17
2.1.1 Khái niệm và phân loại 17
2.1.2 Trợ lý ảo và kiến trúc hội thoại 17
2.2 Mô hình ngôn ngữ lớn 18
2.2.1 Kiến trúc Transformer 18
2.2.2 Tokenization và biểu diễn từ 18
2.2.3 Quá trình huấn luyện 19
2.2.4 Sinh văn bản và prompt 19
2.3 RAG (Retrieval-Augmented Generation) 20
2.3.1 Khái niệm và động lực 20
2.3.2 Kiến trúc tổng quát 20
2.3.3 Chunking và tiền xử lý tài liệu 21
2.3.4 Embedding vector và vector database 21
2.3.5 Sparse retrieval, dense retrieval và hybrid retrieval . . . 21
2.3.6 Rerank 22
2.3.7 Các biến thể của RAG 22
2.4 Fine-tuning mô hình ngôn ngữ lớn (Fine-tuning) 23

2.4.1 Khái niệm và các mức độ . . . 23
2.4.2 Fine-tuning có giám sát và căn chỉnh . . . 23
2.4.3 LoRA . . . 23
2.4.4 QLoRA . . . 24
2.4.5 Khi nào nên dùng fine-tuning . . . 24
2.5 So sánh RAG và fine-tuning . . . 24
2.6 Các thước đo đánh giá . . . 25
2.6.1 Thước đo retrieval . . . 25
2.7 Nền tảng triển khai . . . 26
2.7.1 Phục vụ inference với vLLM . . . 26
2.7.2 Quantization FP8 . . . 26
2.7.3 Giao thức MCP và A2A . . . 26
2.7.4 Quan sát hệ thống và quản lý prompt . . . 26
Kết chương . . . 26

3 Kiến trúc hệ thống đề xuất 28
3.1 Đặc thù bài toán tại Vietnam Post . . . 28
3.1.1 Đặc điểm dữ liệu nội bộ . . . 28
3.1.2 Các nhóm nghiệp vụ . . . 29
3.2 Yêu cầu chức năng và phi chức năng . . . 29
3.2.1 Yêu cầu chức năng . . . 29
3.2.2 Yêu cầu phi chức năng . . . 29
3.3 Kiến trúc tổng thể multi-agent . . . 29
3.4 Pipeline RAG chi tiết . . . 30
3.4.1 Pha indexing . . . 30
3.4.2 Pha retrieval và sinh . . . 31
3.4.3 Tham số hóa và cấu hình . . . 31
3.5 Pipeline fine-tuning đề xuất . . . 31
3.5.1 Thu thập và chuẩn bị dữ liệu . . . 33
3.5.2 Fine-tuning LLM (LoRA/QLoRA) . . . 33
3.5.3 Fine-tuning embedding model (đã thí điểm) . . . 33
3.5.4 Fine-tuning reranker . . . 34
3.6 Kiến trúc hybrid RAG + Fine-tuning . . . 35
3.7 Hạ tầng và bảo mật . . . 36
3.7.1 Hạ tầng phục vụ . . . 36
3.7.2 Bảo mật dữ liệu . . . 36
3.8 Kiến trúc quan sát: OTel Gateway → Langfuse . . . 37

3.8.1 Nguyên tắc thiết kế . . . 37
3.8.2 Pipeline gửi tới Langfuse . . . 37
3.8.3 Định danh phiên và người dùng . . . 38
3.9 Triển khai trên Kubernetes . . . 38
3.9.1 Tổ chức manifests . . . 38
3.9.2 Workload GPU và init container . . . 38
3.9.3 Cấu hình, bản mật và registry . . . 39
Kết chương . . . 39

4 Triển khai thực nghiệm 40
4.1 Môi trường và công cụ . . . 40
4.2 Dữ liệu . . . 41
4.2.1 Các kho tài liệu . . . 41
4.2.2 Bộ mẫu đánh giá retrieval . . . 42
4.2.3 Tiền xử lý . . . 42
4.3 Cài đặt RAG . . . 42
4.3.1 Chunking và embedding . . . 42
4.3.2 Cấu hình retrieval . . . 42
4.3.3 Embedding câu hỏi và prompt . . . 42
4.4 Thiết kế fine-tuning . . . 43
4.4.1 Dữ liệu thí điểm cho embedding model . . . 43
4.4.2 Cấu hình huấn luyện và đánh giá . . . 43
4.4.3 Đóng gói và phục vụ . . . 44
4.4.4 Đề xuất cho LLM và reranker . . . 44
4.5 Triển khai multi-agent . . . 46
4.5.1 Agent điều phối . . . 46
4.5.2 Các agent chuyên biệt và công cụ . . . 46
4.5.3 Quy tắc định tuyến . . . 46
4.6 Quan sát với OTel Gateway . . . 47
4.6.1 Cấu hình OTel Gateway . . . 47
4.6.2 Attribute agent phát . . . 48
4.6.3 Ánh xạ sang Langfuse ở collector . . . 48
4.6.4 Quy trình sử dụng . . . 49
4.7 Triển khai hệ thống lên Kubernetes . . . 50
4.7.1 Tổ chức manifests . . . 50
4.7.2 Workload GPU và init container . . . 51
4.7.3 Cấu hình, bản mật và kết nối nội bộ . . . 51

4.8 Khó khăn và giải pháp 51
Kết chương 51

5 Kết quả và đánh giá 53
5.1 Phương pháp đánh giá 53
5.2 Kết quả khâu retrieval 54
5.2.1 Đã thực hiện được 54
5.2.2 Nhận định rút ra 54
5.3 Kết quả fine-tune embedding model (thí điểm) 55
5.3.1 Phân tích quá trình huấn luyện 56
5.3.2 Kiểm chứng đóng gói 56
5.4 So sánh các hướng tiếp cận 57
5.5 Hiệu năng hệ thống 57
5.6 Case study hội thoại 58
5.7 Hiệu quả vận hành 58
5.8 Thảo luận 59
5.8.1 Ưu điểm 59
5.8.2 Hạn chế 59
5.8.3 Tính khái quát 60
Kết chương 60

6 Kết luận và hướng phát triển 61
6.1 Kết luận 61
6.2 Hạn chế 62
6.3 Hướng phát triển 62
Kết chương 63

A Prompt hệ thống mẫu 64
A.1. Prompt agent điều phối 64
A.2. Prompt agent RAG 64

B Bảng cấu hình tham số 66

C Danh mục mô hình và thành phần 67

D Mẫu hội thoại minh họa 68

Danh sách hình vẽ

2.1 Sơ đồ khối một khối Transformer (một lớp) với self-attention và feed-forward. 19
2.2 Kiến trúc tổng quát của một hệ thống RAG. 20
2.3 Pipeline indexing tài liệu trong hệ thống RAG. 21
2.4 Pipeline hybrid retrieval kết hợp BM25, dense retrieval và rerank. 22
2.5 Minh họa LoRA: giữ nguyên W_0 và thêm cập nhật hạng thấp BA. 24

3.1 Kiến trúc multi-agent: host điều phối các agent chuyên biệt qua A2A. 30
3.2 Pipeline RAG đầy đủ từ indexing đến sinh câu trả lời. 32
3.3 Pipeline fine-tuning đề xuất cho ba loại mô hình: LLM, embedding và reranker. 35
3.4 Kiến trúc hybrid: router chọn RAG, mô hình fine-tuning hoặc kết hợp. 36
3.5 Kiến trúc quan sát: agent phát vết qua OTel Gateway tới Langfuse. 38
3.6 Topologies triển khai trên Kubernetes: workload GPU, workload agent và quan sát. 39

4.1 Giao diện Langfuse và trợ lý AI tích hợp, dùng để quan sát và phân tích phiên hội thoại. 41
4.2 Giao diện trợ lý ảo MiPo: khung hội thoại, danh sách phiên và các nhóm nghiệp vụ được hỗ trợ. 47
4.3 Danh sách trace trên Langfuse (lọc `isRootObservation=true`): mỗi dòng là một lượt `/chat`, kèm agent xử lý và độ trễ. 49
4.4 Chi tiết một trace: các bước `generate-response`, `query_knowledge_base` và lời gọi agent, kèm nội dung hội thoại và trích dẫn ở panel phải. 50

5.1 Đường cong huấn luyện thí điểm. Hàm mất mát giảm đơn điệu và các chỉ số bão hoà từ khoảng bước 60 (epoch 0,8). 55
5.2 So sánh mô hình nền và mô hình sau fine-tune trên bốn chỉ số chính. 56

Danh sách bảng

2.1 So sánh RAG và fine-tuning theo các tiêu chí triển khai. 25

4.1 Các công cụ và thành phần chính của hệ thống. 40
4.2 Thống kê các kho tài liệu sử dụng trong thực nghiệm. 41
4.3 Cấu hình tham số retrieval RAG. 43
4.4 Thống kê bộ dữ liệu thí điểm cho embedding model. 44
4.5 Cấu hình huấn luyện và đánh giá embedding model (thí điểm). 45
4.6 Cấu hình LoRA/QLoRA đề xuất cho fine-tuning LLM. 45
4.7 Cấu hình fine-tuning reranker đề xuất. 46
4.8 Attribute chính mà agent phát theo chuẩn OpenTelemetry. . . . 48
4.9 Nhóm dịch vụ triển khai trên Kubernetes. 50
4.10 Khó khăn gặp phải và giải pháp áp dụng. 52

5.1 Kết quả truy xuất trên tập kiểm định TTHC (708 truy vấn / 1.319 đoạn văn bản). 55
5.2 Kiểm chứng artifact đóng gói so với mô hình HF đã gộp. 56
5.3 So sánh RAG, fine-tuning và kiến trúc hybrid trong bối cảnh Vietnam Post. 57

B.1 Cấu hình tham số tổng hợp của hệ thống. 66

C.1 Các mô hình và thành phần chính được sử dụng. 67

Chương 1

Giới thiệu

1.1 Bối cảnh và lý do chọn đề tài

Trong những năm gần đây, chuyển đổi số đã trở thành một trong những định hướng chiến lược trọng tâm của ngành bưu chính – viễn thông Việt Nam. Tổng công ty Bưu điện Việt Nam (Tổng công ty Bưu điện Việt Nam, sau đây gọi tắt là Vietnam Post) vận hành một mạng lưới rộng khắp với hàng nghìn điểm phục vụ, hàng chục nghìn lao động và một danh mục dịch vụ đa dạng: bưu chính, chuyển phát nhanh (EMS), logistics, tài chính bưu chính và các dịch vụ công. Song song với hoạt động vận hành, Vietnam Post sở hữu một khối lượng tài liệu nghiệp vụ nội bộ rất lớn, bao gồm quy trình nghiệp vụ, quyết định, công văn hướng dẫn, biểu phí dịch vụ, quy định về hàng hóa cấm gửi, chính sách bồi thường và các tiêu chuẩn chất lượng phát.

Thực tế vận hành cho thấy việc tra cứu thủ công khối tài liệu này tiêu tốn đáng kể thời gian của cán bộ công nhân viên. Khi một khách hàng hỏi về cước phí EMS, thời gian toàn trình, quy định về hàng hóa đặc biệt hay chính sách bồi thường, nhân viên phải mở nhiều văn bản khác nhau, tìm đúng điều khoản, rồi tổng hợp câu trả lời. Các kênh hỏi đáp truyền thống như danh mục câu hỏi thường gặp (FAQ) hoặc tổng đài chỉ có thể đáp ứng một phần nhu cầu: FAQ cứng nhắc và nhanh lỗi thời, còn tổng đài phụ thuộc vào số lượng nhân viên và thời gian làm việc. Hệ quả là trải nghiệm khách hàng bị ảnh hưởng và chi phí vận hành tăng lên.

Sự phát triển của các LLM (Large Language Model) mở ra một hướng giải quyết mới. Với khả năng hiểu ngôn ngữ tự nhiên và sinh văn bản mạch lạc, LLM có thể đóng vai trò lõi của một trợ lý ảo có khả năng hội thoại tự nhiên. Tuy nhiên, bản thân một LLM tổng quát không thể trả lời chính xác về tài liệu nội bộ của một tổ chức cụ thể: nó không được huấn luyện trên dữ liệu đó, dễ sinh thông tin sai lệch (hiện tượng “hallucination”), và không thể trích dẫn nguồn để người dùng kiểm chứng. Hai hướng tiếp cận chính được cộng đồng nghiên cứu và công nghiệp áp dụng để khắc phục hạn chế này là:

- **RAG (Retrieval-Augmented Generation – RAG):** tách tri thức ra khỏi tham số mô hình, lấy các document chunk liên quan từ một kho tri thức bên ngoài tại thời điểm inference, rồi đưa chúng vào ngữ cảnh để mô hình sinh câu trả lời có căn cứ và có thể trích nguồn.
- **Fine-tuning (fine-tuning):** tiếp tục huấn luyện mô hình trên dữ liệu chuyên ngành để điều chỉnh hành vi, phong cách và kiến thức miền, giúp mô hình trả lời đúng "giọng" nghiệp vụ và định dạng mong muốn.

Đối với một tổ chức như Vietnam Post, bài toán đặt ra không chỉ là kỹ thuật mà còn là bảo mật dữ liệu: tài liệu nghiệp vụ nội bộ và thông tin khách hàng không được phép rời khỏi hạ tầng của Tổng công ty. Điều này dẫn tới yêu cầu triển khai mô hình cục bộ (on-premise) thay vì gọi các dịch vụ đám mây bên ngoài. Báo cáo này xuất phát từ chính những yêu cầu thực tiễn đó: nghiên cứu và triển khai một trợ lý ảo thông minh cho Vietnam Post, kết hợp RAG và fine-tuning, có khả năng trả lời chính xác, ngữ cảnh hóa dựa trên tài liệu nội bộ, vận hành an toàn trên hạ tầng riêng và có thể mở rộng theo nhu cầu nghiệp vụ.

1.2 Mục tiêu nghiên cứu

Đề tài hướng tới các mục tiêu sau:

1. Hệ thống hóa cơ sở lý thuyết về chatbot/trợ lý ảo, mô hình ngôn ngữ lớn, kỹ thuật RAG và kỹ thuật fine-tuning, làm rõ ưu nhược điểm và phạm vi áp dụng của từng hướng tiếp cận.
2. Thiết kế một kiến trúc trợ lý ảo multi-agent phù hợp với đặc thù dữ liệu và nghiệp vụ của Vietnam Post, trong đó các agent chuyên biệt đảm nhận các miền nghiệp vụ khác nhau và giao tiếp qua các giao thức chuẩn hóa.
3. Triển khai và thực nghiệm một pipeline RAG lai kết hợp sparse retrieval BM25 và dense retrieval trên vector database, có bước rerank, nhằm tạo câu trả lời bám sát tài liệu.
4. Đề xuất pipeline fine-tuning cho cả ba loại mô hình – LLM (LoRA/QLoRA), embedding model và reranker – trong đó thí điểm huấn luyện và đánh giá thật embedding model; cùng tiêu chí quyết định khi nào nên dùng RAG, khi nào nên dùng mô hình đã fine-tuning, và cách kết hợp hai hướng tiếp cận trong một kiến trúc hybrid.

5. Đánh giá hệ thống bằng các thước đo retrieval định lượng (Recall@k, MRR) và định tính (case study hội thoại), từ đó rút ra bài học triển khai và đề xuất lộ trình phát triển.

1.3 Đối tượng và phạm vi nghiên cứu

1.3.1 Đối tượng nghiên cứu

Đối tượng nghiên cứu của đề tài là các phương pháp và kiến trúc triển khai trợ lý ảo dựa trên mô hình ngôn ngữ lớn cho miền nghiệp vụ bưu chính, cụ thể là kỹ thuật RAG và kỹ thuật fine-tuning, cùng các thành phần hỗ trợ như embedding, vector database, rerank và hạ tầng phục vụ mô hình.

1.3.2 Phạm vi nghiên cứu

Đề tài tập trung vào miền dữ liệu và nghiệp vụ của Vietnam Post, bao gồm:

- **Tài liệu văn thư và quy trình nội bộ:** các quyết định, công văn, quy trình nghiệp vụ (tập thử nghiệm gồm 17 tài liệu văn thư đại diện).
- **Tài liệu sản phẩm – dịch vụ:** biểu phí, quy định hàng hóa cấm gửi, chính sách bồi thường, thời gian toàn trình.
- **Tài liệu dịch vụ chuyển phát nhanh EMS:** các quyết định biểu phí, quy định hàng hóa đặc biệt, chính sách bồi thường EMS.
- **Nghiệp vụ tra cứu:** tra cứu tình trạng bưu gửi, tra cứu thông tin bưu cục, và tra cứu chất lượng phát.

Về mặt kỹ thuật, đề tài giới hạn ở các mô hình mã nguồn mở phục vụ cục bộ, không sử dụng các mô hình thương mại qua API đám mây, nhằm bảo đảm yêu cầu bảo mật dữ liệu nội bộ.

1.4 Phương pháp nghiên cứu

Đề tài sử dụng kết hợp nhiều phương pháp:

1. **Nghiên cứu tài liệu:** khảo sát các công trình khoa học và tài liệu kỹ thuật về Transformer, RAG, fine-tuning, LoRA/QLoRA, các thước đo đánh giá và hạ tầng phục vụ inference, nhằm xây dựng nền tảng lý thuyết và lựa chọn công nghệ.

2. **Phân tích và thiết kế hệ thống:** phân tích yêu cầu nghiệp vụ của Vietnam Post, từ đó thiết kế kiến trúc multi-agent, pipeline RAG và pipeline fine-tuning cùng kiến trúc hybrid.
3. **Thực nghiệm:** triển khai hệ thống thử nghiệm, xây dựng các pipeline xử lý dữ liệu – từ nhận dạng ký tự quang học (OCR), chuẩn hóa tài liệu, chunking, embedding vector, đến hybrid retrieval và rerank; tiến hành đo đạc trên dữ liệu nội bộ.
4. **Đánh giá định lượng:** sử dụng các thước đo retrieval (Recall@k, MRR) và đo hiệu năng inference của hệ thống phục vụ.
5. **Đánh giá định tính:** phân tích các tình huống hội thoại thực tế, đánh giá mức độ hữu ích của câu trả lời và tính khả thi khi triển khai ở quy mô lớn.
6. **Ước lượng hiệu quả:** ước tính lợi ích kinh tế – xã hội dựa trên thời gian tiết kiệm được của người dùng so với quy trình thủ công.

1.5 Đóng góp của đề tài

Các đóng góp chính của đề tài bao gồm:

- **Về lý thuyết:** cung cấp một bức tranh tổng hợp, có hệ thống về RAG và fine-tuning trong bối cảnh triển khai trợ lý ảo doanh nghiệp, đặc biệt nhấn mạnh sự đánh đổi giữa tri thức động và hành vi miền, làm cơ sở cho kiến trúc hybrid.
- **Về kiến trúc:** đề xuất một kiến trúc trợ lý ảo multi-agent kết hợp RAG lai và fine-tuning cho miền bưu chính, có khả năng mở rộng và bảo đảm dữ liệu nội bộ không rời khỏi hạ tầng tổ chức.
- **Về thực nghiệm:** triển khai và đo đạc trên dữ liệu và nghiệp vụ thực tế của Vietnam Post, cung cấp bằng chứng định lượng về hiệu quả của từng thành phần (sparse retrieval, dense retrieval, rerank) và chỉ ra điểm nghẽn cần cải thiện; thí điểm fine-tune embedding model cho truy xuất tiếng Việt, đạt mức cải thiện tương đối khoảng 29–33% trên các chỉ số Accuracy@1, Accuracy@5 và NDCG@10.
- **Về đánh giá:** xây dựng quy trình đánh giá retrieval tái lập đúng đường đi sản xuất kèm phân tách lỗi retrieval/rerank, phù hợp với đặc thù tiếng Việt và miền nghiệp vụ hẹp.
- **Về thực tiễn:** đề xuất lộ trình triển khai và ước lượng hiệu quả kinh

tế, giúp tổ chức ra quyết định đầu tư và mở rộng hệ thống.

1.6 Cấu trúc báo cáo

Ngoài phần mở đầu và phụ lục, báo cáo được tổ chức thành sáu chương:

- **Chương 1 – Giới thiệu:** trình bày bối cảnh, mục tiêu, đối tượng, phạm vi, phương pháp nghiên cứu, đóng góp và cấu trúc báo cáo.
- **Chương 2 – Cơ sở lý thuyết:** hệ thống hóa kiến thức về chatbot, LLM, RAG, fine-tuning, kiến trúc hybrid, thước đo đánh giá và nền tảng triển khai.
- **Chương 3 – Kiến trúc hệ thống đề xuất:** phân tích bài toán, yêu cầu và thiết kế kiến trúc multi-agent, pipeline RAG, pipeline fine-tuning và kiến trúc hybrid cho Vietnam Post.
- **Chương 4 – Triển khai thực nghiệm:** mô tả môi trường, dữ liệu, cấu hình RAG, thiết kế fine-tuning, triển khai multi-agent và hệ thống quan sát.
- **Chương 5 – Kết quả và đánh giá:** trình bày kết quả retrieval, kết quả fine-tune embedding model, so sánh các hướng tiếp cận, hiệu năng hệ thống, case study và hiệu quả kinh tế – xã hội.
- **Chương 6 – Kết luận và hướng phát triển:** tóm tắt kết quả, nêu hạn chế và đề xuất hướng phát triển.

Phần cuối báo cáo gồm tài liệu tham khảo và các phụ lục (prompt mẫu, bảng cấu hình tham số và danh mục mô hình).

Chương 2

Cơ sở lý thuyết và tổng quan

2.1 Chatbot và trợ lý ảo

2.1.1 Khái niệm và phân loại

Chatbot là một chương trình máy tính được thiết kế để tương tác với con người thông qua hội thoại ngôn ngữ tự nhiên. Có thể phân loại chatbot theo nhiều tiêu chí. Theo mức độ phức tạp của cơ chế xử lý, chatbot được chia thành ba thế hệ:

- **Chatbot dựa trên luật (rule-based):** hoạt động theo các kịch bản và mẫu câu lệnh được định nghĩa trước. Ưu điểm là dễ kiểm soát và tiên đoán, nhưng cứng nhắc, khó mở rộng và dễ thất bại trước các cách diễn đạt ngoài kịch bản.
- **Chatbot dựa trên học máy (retrieval-based / intent-based):** sử dụng các mô hình phân loại ý định và trích xuất thực thể để ánh xạ câu hỏi người dùng vào một tập câu trả lời định sẵn. Linh hoạt hơn rule-based nhưng vẫn phụ thuộc vào tập ý định hữu hạn.
- **Chatbot sinh văn bản (generative) dựa trên mô hình ngôn ngữ lớn:** sinh câu trả lời trực tiếp từ mô hình ngôn ngữ, có khả năng diễn đạt tự nhiên và xử lý câu hỏi mở. Đây là hướng tiếp cận mà đề tài lựa chọn.

2.1.2 Trợ lý ảo và kiến trúc hội thoại

Khái niệm “trợ lý ảo” (virtual assistant) thường được hiểu rộng hơn chatbot: không chỉ trả lời câu hỏi mà còn có khả năng thực hiện tác vụ, gọi công cụ, truy cập hệ thống nghiệp vụ và duy trì ngữ cảnh qua nhiều lượt hội thoại. Một trợ lý ảo hiện đại thường gồm các thành phần: (i) bộ hiểu ngôn ngữ và quản lý hội thoại; (ii) bộ truy xuất tri thức; (iii) bộ gọi công cụ (tool calling); (iv) bộ sinh câu trả lời; và (v) lớp quan sát, ghi log để giám sát chất lượng.

Trong các hệ thống lớn, kiến trúc *multi-agent* (multi-agent) ngày càng phổ biến: một agent điều phối (orchestrator/host) tiếp nhận yêu cầu người dùng và định tuyến đến các agent chuyên biệt (specialist agents), mỗi agent phụ trách một miền nghiệp vụ hẹp. Cách tổ chức này giúp hệ thống dễ mở rộng, dễ kiểm thử và giảm tải nhận thức cho từng thành phần. Để các agent có thể giao tiếp và chia sẻ công cụ một cách chuẩn hóa, các giao thức như Model Context Protocol (MCP) và Agent-to-Agent (A2A) được sử dụng (trình bày ở mục 2.7).

2.2 Mô hình ngôn ngữ lớn

2.2.1 Kiến trúc Transformer

Hầu hết các mô hình ngôn ngữ lớn hiện đại đều dựa trên kiến trúc Transformer [1]. Khác với mạng hồi tiếp (RNN) xử lý tuần tự từng token, Transformer xử lý toàn bộ chuỗi song song nhờ cơ chế *tự chú ý* (self-attention), cho phép mô hình nắm bắt quan hệ giữa các token ở xa nhau một cách hiệu quả và song song hóa tốt trên GPU.

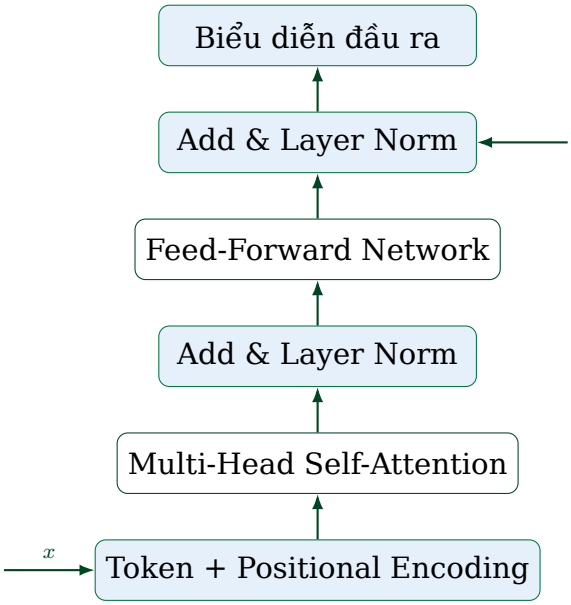
Cơ chế tự chú ý được mô tả bằng công thức *scaled dot-product attention*:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V, \tag{2.1}$$

trong đó Q, K, V lần lượt là các ma trận truy vấn (query), khóa (key) và giá trị (value) được chiếu tuyến tính từ biểu diễn đầu vào, và d_k là số chiều của vector khóa. Cơ chế *đa đầu* (multi-head attention) chạy song song nhiều phép chú ý trên các không gian con khác nhau rồi ghép kết quả, giúp mô hình học nhiều loại quan hệ đồng thời. Mỗi khối Transformer còn gồm mạng truyền thẳng theo vị trí (position-wise feed-forward network), chuẩn hóa lớp (layer normalization) và kết nối tắt (residual connection). Thông tin vị trí của token được bổ sung qua positional encoding.

2.2.2 Tokenization và biểu diễn từ

Trước khi đưa vào mô hình, văn bản được chia thành các *token* bằng bộ tokenizer (thường theo thuật toán BPE [2] hoặc SentencePiece). Mỗi token được ánh xạ thành một vector embedding; các vector này được cập nhật qua nhiều lớp Transformer để tạo ra biểu diễn ngữ cảnh hóa. Chất lượng



Hình 2.1: Sơ đồ khối một khối Transformer (một lớp) với self-attention và feed-forward.

tokenizer ảnh hưởng trực tiếp đến khả năng xử lý tiếng Việt, đặc biệt với các từ ghép và dấu thanh.

2.2.3 Quá trình huấn luyện

Một mô hình ngôn ngữ lớn thường trải qua các giai đoạn:

1. **Tiền huấn luyện (pre-training):** huấn luyện mô hình dự đoán token tiếp theo trên khối lượng văn bản khổng lồ [3]. Giai đoạn này giúp mô hình học ngữ pháp, kiến thức phổ quát và khả năng lập luận ngôn ngữ.
2. **Fine-tuning có giám sát (instruction tuning/SFT):** tiếp tục huấn luyện trên các cặp hướng dẫn – đáp án để mô hình biết cách "trả lời" thay vì chỉ "hoàn thành văn bản" [4].
3. **Căn chỉnh theo phản hồi con người (RLHF) hoặc tối ưu theo sở thích (DPO):** điều chỉnh mô hình theo sở thích của con người về tính hữu ích, trung thực và an toàn [5, 6].

2.2.4 Sinh văn bản và prompt

Khi inference, mô hình sinh token lần lượt theo một chiến lược giải mã. Nhiệt độ (temperature) điều chỉnh độ ngẫu nhiên; top-k và top-p (nucleus sampling) giới hạn tập token được chọn, giúp cân bằng giữa tính đa dạng và

tính ổn định. Chất lượng đầu ra phụ thuộc rất lớn vào *prompt*: kỹ thuật zero-shot, few-shot (đưa ví dụ) và Chain-of-Thought (CoT) [7] giúp định hướng mô hình giải quyết tác vụ.

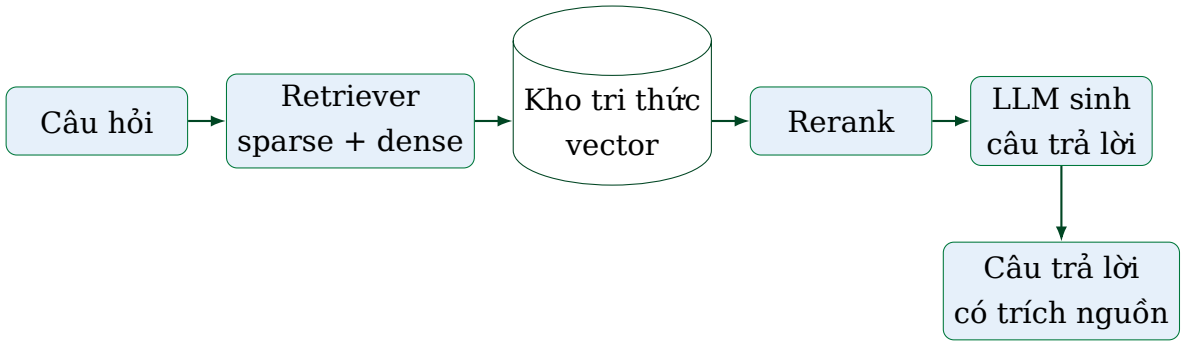
2.3 RAG (Retrieval-Augmented Generation)

2.3.1 Khái niệm và động lực

RAG [8] là kỹ thuật kết hợp một retriever với một mô hình sinh (generator). Thay vì buộc mô hình phải "nhớ" toàn bộ tri thức trong tham số, RAG lấy các document chunk liên quan từ kho tri thức ngoài tại thời điểm inference, rồi cung cấp chúng làm ngữ cảnh để mô hình sinh câu trả lời. Cách tiếp cận này mang lại ba lợi ích chính: (i) cập nhật tri thức mà không cần huấn luyện lại; (ii) giảm hiện tượng hallucination nhờ ràng buộc vào bằng chứng; và (iii) cho phép trích dẫn nguồn, tăng tính kiểm chứng.

2.3.2 Kiến trúc tổng quát

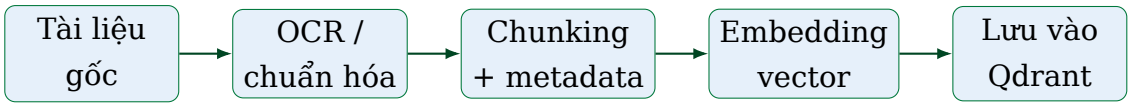
Một hệ RAG điển hình gồm hai pha: *indexing* và *inference*. Ở pha indexing, tài liệu được chuẩn hóa, chia thành chunk, embedding thành vector và lưu vào vector database. Ở pha inference, câu hỏi của người dùng được embedding, retriever tìm các chunk liên quan, chúng được rerank và ghép thành ngữ cảnh, cuối cùng mô hình sinh câu trả lời.



Hình 2.2: Kiến trúc tổng quát của một hệ thống RAG.

2.3.3 Chunking và tiền xử lý tài liệu

Chất lượng của RAG phụ thuộc trước hết vào cách tài liệu được chia thành các *chunk*. Chunk quá lớn làm loãng ngữ cảnh và tăng chi phí; chunk quá nhỏ làm mất mạch thông tin. Với tài liệu nghiệp vụ chứa nhiều bảng biểu và tiêu đề, chiến lược chunking thường dùng là *nhận biết cấu trúc* (header-aware): cắt theo tiêu đề mục, giữ nguyên bảng, đặt độ dài tối đa và độ chồng lấn (overlap) hợp lý để bảo toàn ngữ cảnh giữa các chunk liền kề. Với tài liệu scan hoặc ảnh, cần bước nhận dạng ký tự quang học (OCR) trước khi chunking.



Hình 2.3: Pipeline indexing tài liệu trong hệ thống RAG.

2.3.4 Embedding vector và vector database

Mô hình embedding ánh xạ text chunk thành một vector số thực trong không gian nhiều chiều, sao cho các chunk có ngữ nghĩa gần nhau nằm gần nhau. Độ tương đồng thường được đo bằng cosine:

$$\mathrm{sim}(\mathbf{u}, \mathbf{v}) = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\| \, \|\mathbf{v}\|}. \tag{2.2}$$

Vector database lưu trữ và tìm kiếm láng giềng gần nhất trên hàng triệu vector một cách hiệu quả nhờ các thuật toán chỉ mục xấp xỉ [9]. Trong đề tài này, Qdrant [10] được sử dụng với khả năng lưu song song dense vector (dense) và sparse vector (sparse) trong cùng một collection.

2.3.5 Sparse retrieval, dense retrieval và hybrid retrieval

Sparse retrieval biểu diễn văn bản bằng sparse vector dựa trên tần suất từ, tiêu biểu là BM25 [11]. Điểm BM25 giữa câu truy vấn q và tài liệu d được tính:

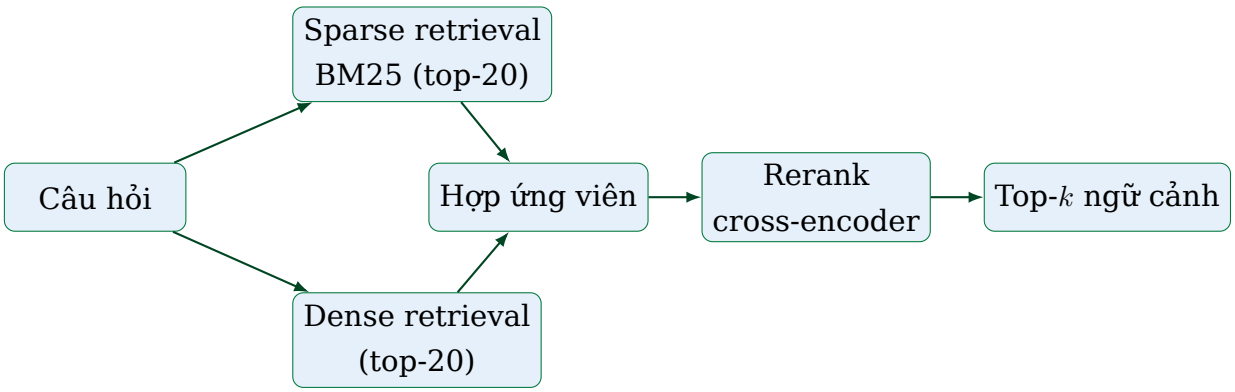
$$\mathrm{BM25}(q, d) = \sum_{t \in q} \mathrm{IDF}(t) \cdot \frac{f(t, d)\,(k_1 + 1)}{f(t, d) + k_1 \left(1 - b + b\,\frac{|d|}{\mathrm{avgdl}}\right)}, \tag{2.3}$$

trong đó $f(t, d)$ là tần suất từ t trong d, $|d|$ là độ dài tài liệu, avgdl là độ dài trung bình, và k_1, b là các tham số điều chỉnh. BM25 mạnh ở việc khớp từ khóa chính xác, đặc biệt với số hiệu văn bản, mã dịch vụ hoặc thuật ngữ hiếm.

Dense retrieval [12] sử dụng embedding model để biểu diễn câu hỏi và tài liệu trong cùng không gian ngữ nghĩa, nhờ đó bắt được các trường hợp diễn đạt đồng nghĩa mà BM25 bỏ sót. Hai phương pháp có ưu nhược điểm bổ trợ cho nhau. *Hybrid retrieval* kết hợp cả hai bằng cách lấy hợp hoặc kết hợp điểm số, thường cho kết quả tốt hơn từng phương pháp riêng lẻ.

2.3.6 Rerank

Các retriever giai đoạn đầu (first-stage retrieval) thường nhanh nhưng kém chính xác do phải quét trên toàn kho. Bộ *reranker* sắp xếp lại thứ tự ứng viên bằng một mô hình mạnh hơn – thường là mô hình cross-encoder đọc đồng thời câu hỏi và chunk để cho điểm tương quan. Quy trình điển hình: lấy một tập ứng viên lớn ở giai đoạn một (ví dụ top-20 mỗi nhánh), hợp chúng lại, rồi rerank toàn bộ và giữ top-k nhỏ (ví dụ top-5) làm ngữ cảnh cho mô hình sinh.



Hình 2.4: Pipeline hybrid retrieval kết hợp BM25, dense retrieval và rerank.

2.3.7 Các biến thể của RAG

Từ kiến trúc cơ bản, RAG có nhiều biến thể [13]. *Naive RAG* là dạng đơn giản nhất với truy xuất một lần. *Advanced RAG* bổ sung tiền xử lý truy vấn, hậu xử lý kết quả và rerank. *Modular RAG* tổ chức thành các mô-đun linh hoạt có thể thay thế. Ngoài ra, *GraphRAG* khai thác đồ thị tri thức để truy xuất theo quan hệ giữa các thực thể, và *agentic RAG* cho phép agent quyết

định truy xuất nhiều vòng, cải viết truy vấn hoặc gọi công cụ khác khi cần. Đề tài này áp dụng advanced/modular RAG với hybrid retrieval và rerank, đồng thời định hướng mở rộng lên agentic và GraphRAG trong tương lai.

2.4 Fine-tuning mô hình ngôn ngữ lớn (Fine-tuning)

2.4.1 Khái niệm và các mức độ

Fine-tuning (fine-tuning) là quá trình tiếp tục huấn luyện một mô hình đã được tiền huấn luyện trên một dataset chuyên biệt, nhằm điều chỉnh mô hình phù hợp với miền hoặc tác vụ cụ thể. Có thể chia thành hai nhóm:

- **Full fine-tuning:** cập nhật toàn bộ tham số mô hình. Hiệu quả cao nhưng đòi hỏi tài nguyên tính toán và bộ nhớ rất lớn, đồng thời cần lưu toàn bộ bản sao mô hình cho mỗi tác vụ.
- **Fine-tuning hiệu quả tham số (PEFT):** chỉ cập nhật một phần nhỏ tham số, giữ nguyên phần lớn mô hình gốc. Tiêu biểu là LoRA và QLoRA.

2.4.2 Fine-tuning có giám sát và căn chỉnh

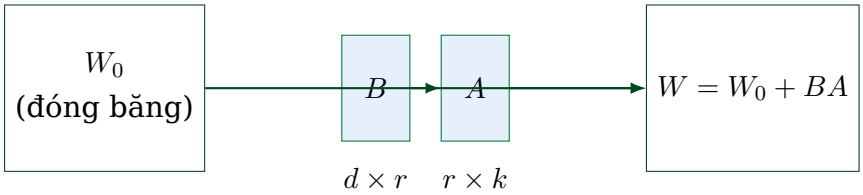
Fine-tuning có giám sát (Supervised Fine-Tuning – SFT) huấn luyện mô hình trên các cặp hướng dẫn – đáp án chất lượng cao. Để mô hình trả lời đúng "giọng" nghiệp vụ, dữ liệu SFT cần được thiết kế cẩn thận cả về nội dung lẫn định dạng. Sau SFT, có thể áp dụng RLHF hoặc DPO [6] để căn chỉnh theo sở thích con người, giúp câu trả lời hữu ích và an toàn hơn.

2.4.3 LoRA

LoRA (Low-Rank Adaptation) [14] giả định rằng thay đổi tham số khi thích ứng một tác vụ có "hạng thấp" (low intrinsic rank). Thay vì cập nhật toàn bộ ma trận trọng số $W_0 \in \mathbb{R}^{d\times k}$, LoRA giữ nguyên W_0 và thêm một phần hiệu chỉnh phân rã thành tích hai ma trận hạng thấp:

$$W = W_0 + \Delta W = W_0 + BA, \qquad B \in \mathbb{R}^{d\times r},\ A \in \mathbb{R}^{r\times k},\ r \ll \min(d,k). \tag{2.4}$$

Ma trận A được khởi tạo ngẫu nhiên và B được khởi tạo bằng 0 để lúc bắt đầu $\Delta W = 0$, bảo toàn hành vi mô hình gốc. Số tham số huấn luyện giảm từ $d \times k$ xuống $r(d+k)$, thường chỉ còn dưới 1% tổng tham số. Hệ số α và hạng r quyết định độ lớn của cập nhật; giá trị hiệu chỉnh thường được nhân với α/r.



Hình 2.5: Minh họa LoRA: giữ nguyên W_0 và thêm cập nhật hạng thấp BA.

2.4.4 QLoRA

QLoRA [15] mở rộng LoRA bằng cách quantization mô hình gốc xuống 4 bit (định dạng NF4), áp dụng quantization kép (double quantization) và tối ưu hóa phân trang (paged optimizers). Nhờ đó có thể fine-tuning các mô hình lớn trên phần cứng khiêm tốn hơn đáng kể so với full fine-tuning, với mức suy giảm chất lượng nhỏ.

2.4.5 Khi nào nên dùng fine-tuning

Fine-tuning phù hợp khi cần điều chỉnh *hành vi* của mô hình (phong cách, định dạng, quy trình nghiệp vụ ổn định) hoặc khi tri thức miền mang tính ổn định và khó mô tả bằng ngữ cảnh retrieval. Ngược lại, khi tri thức thay đổi thường xuyên hoặc cần trích dẫn nguồn, RAG là lựa chọn phù hợp hơn.

2.5 So sánh RAG và fine-tuning

RAG và fine-tuning giải quyết hai lớp vấn đề khác nhau và có thể bổ trợ cho nhau. Bảng 2.1 tóm tắt sự so sánh.

Từ so sánh trên, đề tài đề xuất *kiến trúc hybrid* kết hợp cả hai: sử dụng RAG làm kênh cung cấp tri thức động và bằng chứng, đồng thời dùng mô hình đã fine-tuning để sinh câu trả lời đúng phong cách, đúng định dạng nghiệp vụ và xử lý các tình huống ổn định. Một router quyết định khi nào

Bảng 2.1: So sánh RAG và fine-tuning theo các tiêu chí triển khai.

Tiêu chí	**RAG**	**Fine-tuning**
Nguồn tri thức	Kho tài liệu ngoài, truy xuất động	Tham số mô hình, cố định sau huấn luyện
Cập nhật tri thức	Chỉ cần cập nhật chỉ mục	Cần huấn luyện lại
Khả năng trích nguồn	Có (dựa trên chunk retrieval)	Hạn chế
Chi phí huấn luyện	Thấp (chủ yếu indexing)	Cao hơn (SFT/LoRA)
Chi phí inference	Cao hơn (thêm retrieval)	Thấp hơn (chỉ sinh)
Điều chỉnh phong cách/định dạng	Yếu	Mạnh
Giảm hallucination	Tốt nếu truy xuất đúng	Trung bình
Phù hợp nhất với	Tri thức động, tra cứu	Hành vi miền ổn định

chỉ cần RAG, khi nào dùng mô hình fine-tuning, và khi nào kết hợp. Chi tiết kiến trúc được trình bày ở chương 3.

2.6 Các thước đo đánh giá

Đánh giá một hệ RAG tập trung trước hết vào chất lượng retrieval – khâu quyết định câu trả lời có đủ căn cứ hay không.

2.6.1 Thước đo retrieval

- **Recall@k:** tỉ lệ các tài liệu/chunk liên quan được tìm thấy trong top-k. Đây là thước đo quan trọng nhất vì nếu tài liệu đúng không nằm trong ngữ cảnh thì mô hình không thể trả lời đúng.
- **Precision@k:** tỉ lệ chunk liên quan trong top-k.
- **MRR (Mean Reciprocal Rank):** nghịch đảo thứ hạng của chunk liên quan đầu tiên, trung bình trên các truy vấn. MRR cao nghĩa là chunk đúng xuất hiện sớm.
- **nDCG:** thước đo có trọng số theo vị trí, phù hợp khi đánh giá chất lượng xếp hạng với mức độ liên quan khác nhau.

2.7 Nền tảng triển khai

2.7.1 Phục vụ inference với vLLM

vLLM [16, 17] là thư viện LLM serving LLM hiệu năng cao, nổi bật với cơ chế PagedAttention quản lý bộ nhớ KV-cache theo trang, cho phép tăng throughput đáng kể nhờ xử lý theo lô liên tục (continuous batching). Kết hợp với quantization FP8, vLLM cho phép phục vụ các mô hình hàng chục tỷ tham số trên GPU với chi phí hợp lý.

2.7.2 Quantization FP8

FP8 là định dạng số dấu phẩy động 8 bit, giảm một nửa dung lượng bộ nhớ so với FP16 và tăng tốc tính toán trên phần cứng hỗ trợ, với mức suy giảm chất lượng thường nhỏ. Đây là lựa chọn phổ biến để phục vụ mô hình lớn trên hạ tầng giới hạn.

2.7.3 Giao thức MCP và A2A

Model Context Protocol (MCP) [18] chuẩn hóa cách mô hình truy cập công cụ và nguồn dữ liệu, giúp tách biệt logic công cụ khỏi agent. **Agent-to-Agent (A2A)** [19] chuẩn hóa giao tiếp giữa các agent, cho phép một agent điều phối gọi agent chuyên biệt và nhận kết quả. Hai giao thức này tạo nên nền tảng cho kiến trúc multi-agent của đề tài.

2.7.4 Quan sát hệ thống và quản lý prompt

OpenTelemetry (OTel) [20] cung cấp chuẩn thu thập vết (trace), chỉ số (metric) và log; dữ liệu được đẩy tới Langfuse [21] để giám sát inference flow, latency và lượng token. MLflow Prompt Registry [22] cho phép quản lý phiên bản prompt tập trung, giúp tách cấu hình khỏi mã nguồn và kiểm soát thay đổi prompt theo môi trường (ví dụ nhãn `@production`).

Kết chương

Chương này đã hệ thống hóa nền tảng lý thuyết về chatbot, LLM, RAG, fine-tuning, kiến trúc lai, thước đo đánh giá và nền tảng triển khai. Đây là cơ sở

để chương tiếp theo phân tích bài toán cụ thể của Vietnam Post và thiết kế kiến trúc hệ thống đề xuất.

Chương 3

Kiến trúc hệ thống đề xuất

Trên nền tảng lý thuyết đã trình bày ở chương 2, chương này phân tích bài toán cụ thể của Vietnam Post và đề xuất kiến trúc trợ lý ảo. Mục 3.1 làm rõ đặc thù bài toán; mục 3.2 xác định yêu cầu; mục 3.3 trình bày kiến trúc multi-agent; mục 3.4 và mục 3.5 mô tả pipeline RAG và pipeline fine-tuning; mục 3.6 đề xuất kiến trúc hybrid; mục 3.7 trình bày hạ tầng phục vụ mô hình và bảo mật; mục 3.8 trình bày kiến trúc quan sát dựa trên OTel Gateway; và mục 3.9 trình bày triển khai trên Kubernetes.

3.1 Đặc thù bài toán tại Vietnam Post

3.1.1 Đặc điểm dữ liệu nội bộ

Dữ liệu nội bộ của Vietnam Post có một số đặc điểm ảnh hưởng trực tiếp đến thiết kế hệ thống:

- **Đa dạng định dạng:** tài liệu tồn tại dưới dạng văn bản số (docx, pdf, txt) và cả bản scan. Điều này đòi hỏi bước nhận dạng ký tự quang học và chuẩn hóa về một định dạng thống nhất trước khi indexing.
- **Cấu trúc phức tạp:** nhiều văn bản chứa bảng biểu (biểu phí, thời gian toàn trình, bảng phân loại hàng hóa), tiêu đề nhiều cấp và số hiệu văn bản. Việc chunking phải bảo toàn bảng và ngữ cảnh tiêu đề.
- **Ngôn ngữ chuyên ngành:** nhiều thuật ngữ, từ viết tắt và số hiệu đặc thù (ví dụ số quyết định, mã dịch vụ), đòi hỏi cấu hình sparse retrieval phù hợp với tiếng Việt.
- **Tính động:** biểu phí và quy định thay đổi theo thời gian, nên tri thức cần được cập nhật mà không phải huấn luyện lại mô hình.
- **Tính bảo mật:** tài liệu nội bộ và dữ liệu khách hàng không được phép rời khỏi hạ tầng của Tổng công ty.

3.1.2 Các nhóm nghiệp vụ

Hệ thống phục vụ ba nhóm nghiệp vụ chính: (i) hỏi đáp trên tài liệu văn thư và sản phẩm – dịch vụ; (ii) tra cứu tình trạng bưu gửi; và (iii) tra cứu thông tin bưu cục. Ngoài ra còn các nghiệp vụ mở rộng như tra cứu chất lượng phát và tra cứu tài chính – kế toán.

3.2 Yêu cầu chức năng và phi chức năng

3.2.1 Yêu cầu chức năng

- Trả lời câu hỏi tự nhiên dựa trên tài liệu nội bộ, có trích dẫn nguồn.
- Ngữ cảnh hóa câu trả lời theo lịch sử hội thoại nhiều lượt.
- Tra cứu tình trạng bưu gửi và thông tin bưu cục qua kết nối hệ thống nghiệp vụ.
- Định tuyến yêu cầu đến agent chuyên biệt phù hợp và hợp nhất kết quả.
- Hỗ trợ tạo phiếu yêu cầu (ticket) khi cần chuyển sang hỗ trợ thủ công.
- Gợi ý câu hỏi mở đầu phù hợp với nhu cầu người dùng.

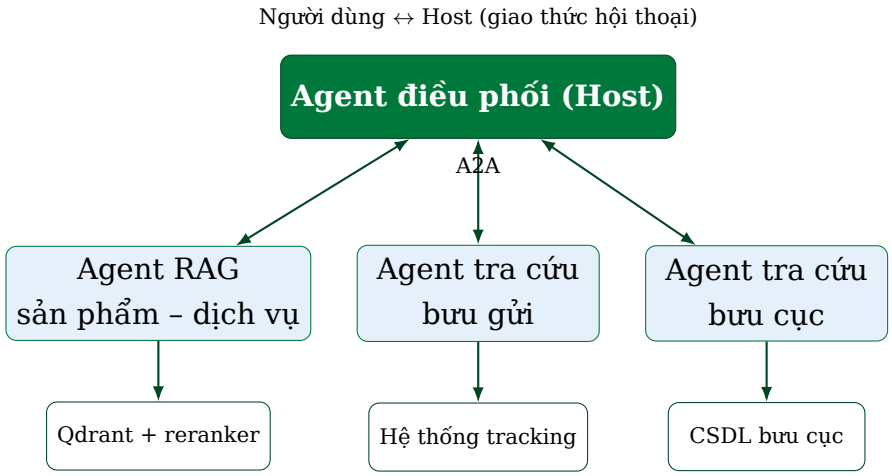
3.2.2 Yêu cầu phi chức năng

- **Chính xác và có căn cứ:** câu trả lời phải bám sát tài liệu, hạn chế tối đa hallucination.
- **Bảo mật:** toàn bộ mô hình và dữ liệu vận hành cục bộ, không gửi ra ngoài.
- **Latency chấp nhận được:** phản hồi trong vài giây cho câu hỏi thông thường.
- **Khả năng mở rộng:** dễ thêm agent, kho tài liệu và mô hình mới.
- **Quan sát được:** có vết, log và chỉ số để giám sát và cải tiến.
- **Kiểm soát truy cập:** phân quyền theo vai trò đối với dữ liệu nhạy cảm.

3.3 Kiến trúc tổng thể multi-agent

Đề tài đề xuất kiến trúc *multi-agent* gồm một agent điều phối trung tâm (host orchestrator) và các agent chuyên biệt. Agent điều phối chịu trách

nhiệm hiểu yêu cầu, duy trì ngữ cảnh hội thoại, quyết định định tuyến và hợp nhất câu trả lời. Các agent chuyên biệt đảm nhận từng miền nghiệp vụ hẹp và được gọi qua giao thức A2A; mỗi agent truy cập công cụ của mình qua giao thức MCP.



Hình 3.1: Kiến trúc multi-agent: host điều phối các agent chuyên biệt qua A2A.

Ưu điểm của kiến trúc này là tách biệt trách nhiệm rõ ràng: mỗi agent có prompt, công cụ và kho dữ liệu riêng, dễ kiểm thử và nâng cấp độc lập. Agent điều phối chỉ cần biết "khi nào gọi ai", nhờ đó giảm tải nhận thức và tăng độ ổn định.

3.4 Pipeline RAG chi tiết

3.4.1 Pha indexing

Pha indexing biến tài liệu thô thành các chunk vector trong Qdrant qua các bước:

1. **Thu thập và chuẩn hóa:** tài liệu được thu thập, với các bản scan được xử lý OCR bằng mô hình nhận dạng (ví dụ `olmocr-2-7b` hoặc `lightonocr2-1b`).
2. **Chuyển sang Markdown:** tài liệu được chuyển thành văn bản có cấu trúc để bảo toàn tiêu đề và bảng.
3. **Trích xuất metadata:** mỗi tài liệu được gắn metadata như tiêu đề, số hiệu văn bản, ngày ban hành và tóm tắt do mô hình ngôn ngữ sinh ra.
4. **Chunking:** văn bản được chia thành các chunk nhận biết cấu trúc, với độ dài tối đa và độ chồng lấn hợp lý, bảo toàn bảng biểu.

5. **Embedding và lưu trữ:** mỗi chunk được biểu diễn thành embedding vector bằng embedding model, rồi lưu vào Qdrant cùng sparse vector cho BM25.

3.4.2 Pha retrieval và sinh

Khi có câu hỏi, hệ thống thực hiện:

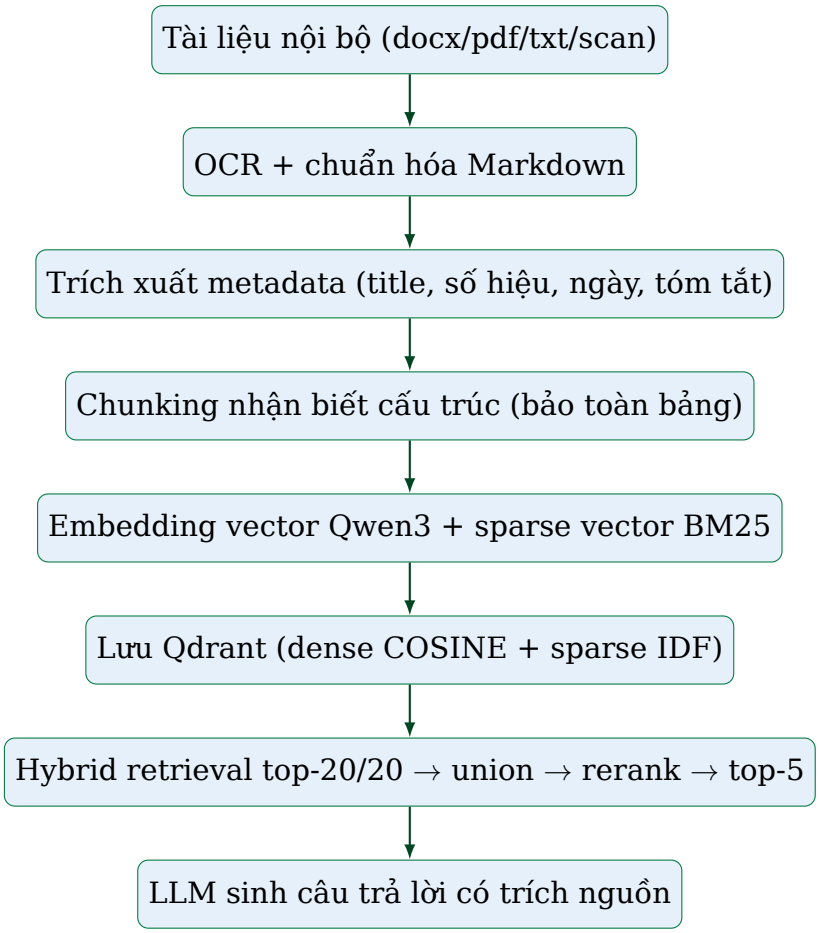
1. **Embedding câu hỏi** bằng cùng embedding model đã dùng khi indexing.
2. **Hybrid retrieval:** chạy song song sparse retrieval BM25 và dense retrieval, mỗi nhánh lấy top-k_1 ứng viên (ví dụ 20).
3. **Hợp ứng viên:** gộp kết quả hai nhánh và loại trùng.
4. **Rerank:** dùng cross-encoder rerank toàn bộ ứng viên và giữ top-k_2 (ví dụ 5) làm ngữ cảnh.
5. **Sinh câu trả lời:** mô hình ngôn ngữ nhận câu hỏi, ngữ cảnh retrieval và lịch sử hội thoại để sinh câu trả lời có trích nguồn.

3.4.3 Tham số hóa và cấu hình

Các tham số chính gồm số ứng viên mỗi nhánh retrieval (k_1), số chunk sau rerank (k_2) và ngưỡng điểm tối thiểu để loại bỏ ngữ cảnh không liên quan. Cấu hình thực tế được trình bày ở chương 4. Cơ chế ngưỡng điểm giúp hệ thống từ chối trả lời khi không tìm thấy bằng chứng phù hợp, thay vì "bịa" câu trả lời.

3.5 Pipeline fine-tuning đề xuất

Trong một hệ RAG, có *ba loại mô hình* có thể được fine-tune với mục tiêu khác nhau: (i) **LLM** sinh câu trả lời – điều chỉnh hành vi, phong cách và định dạng nghiệp vụ; (ii) **embedding model** – cải thiện chất lượng biểu diễn để dense retrieval chính xác hơn; và (iii) **reranker** – cải thiện xếp hạng để giữ đúng chunk trong top-k. Hai loại sau tác động trực tiếp vào khâu retrieval/rerank, vốn là điểm nghẽn được chỉ ra ở chương 5. Trong khi LLM và reranker vẫn dừng ở mức *đề xuất*, pipeline cho *embedding model* đã được *thí điểm triển khai và đánh giá* trên một bộ dữ liệu Thủ tục Hành



Hình 3.2: Pipeline RAG đầy đủ từ indexing đến sinh câu trả lời.

chính (TTHC) tiếng Việt; phần này trình bày thiết kế chung, còn cấu hình và kết quả thí điểm lần lượt được báo cáo ở các chương 4 và 5.

3.5.1 Thu thập và chuẩn bị dữ liệu

Với LLM (đề xuất), dữ liệu huấn luyện được tổng hợp từ ba nguồn: (i) log hội thoại thực tế giữa người dùng và hệ thống, sau khi lọc và ẩn thông tin định danh cá nhân (PII); (ii) các cặp hỏi – đáp chuẩn do chuyên gia nghiệp vụ xây dựng; và (iii) dữ liệu tổng hợp do mô hình mạnh sinh ra có kiểm duyệt. Tùy loại mô hình, dữ liệu được định dạng khác nhau:

- **Với LLM:** định dạng theo mẫu hội thoại (instruction – response).
- **Với embedding và reranker:** định dạng theo cặp truy vấn – chunk, trong đó chunk chứa đáp án là mẫu dương (positive), còn các chunk được truy hồi nhưng không chứa đáp án là mẫu âm (negative). Các mẫu âm *khó* (hard negatives) được khai thác trực tiếp từ kết quả truy hồi sai của BM25/dense, giúp mô hình học phân biệt tốt hơn.

Dữ liệu được chia thành tập huấn luyện, tập kiểm định và tập kiểm thử *theo câu hỏi* để tránh rò rỉ dữ liệu giữa các tập.

3.5.2 Fine-tuning LLM (LoRA/QLoRA)

Đề xuất sử dụng LoRA hoặc QLoRA với hạng $r = 16$, hệ số $\alpha = 32$, dropout nhỏ, và áp dụng lên các ma trận chiếu trong khối attention và feed-forward (query, key, value, output, gate, up, down). Mô hình gốc được giữ ở định dạng quantization 4 bit khi dùng QLoRA để tiết kiệm bộ nhớ. Sau huấn luyện, adapter có thể được gộp (merge) vào mô hình gốc hoặc phục vụ như một adapter riêng trong vLLM. Mục tiêu là điều chỉnh *hành vi* chứ không nạp tri thức (tri thức do RAG đảm nhiệm).

3.5.3 Fine-tuning embedding model (đã thí điểm)

Mục tiêu là tăng Recall@k của dense retrieval bằng cách đưa biểu diễn câu hỏi và chunk chứa đáp án lại gần nhau hơn. Đây là phần duy nhất trong pipeline fine-tuning đã được triển khai thật: một thí điểm trên mô hình nền Qwen3-Embedding-0.6B [23], huấn luyện và đánh giá trên một bộ dữ liệu TTHC tiếng Việt.

Phương pháp là *học tương phản* (contrastive learning) với hàm mất mát *MultipleNegativesRankingLoss* [24]: mỗi cặp truy vấn – chunk dương được đối chiếu với các mẫu âm trong cùng batch (in-batch negatives) và các *hard negative*. Với mỗi truy vấn, hard negative được khai thác bằng chính mô hình nền: chọn tối đa $k = 3$ chunk có cosine similarity cao nhất nhưng không phải positive, loại các trường hợp trùng positive.

Quy trình dữ liệu gồm bốn bước: (i) chuyển văn bản PDF sang Markdown rồi cắt theo cấu trúc mục (*chunking*) và nhúng bằng mô hình nền; (ii) sinh truy vấn tổng hợp bằng LLM cho từng chunk; (iii) khai thác hard negative như trên; và (iv) tạo bộ ba $(\text{query}, \text{positive}, \text{negative})$ và chia tập *theo văn bản* (document-level) để tránh rò rỉ giữa tập huấn luyện và kiểm định.

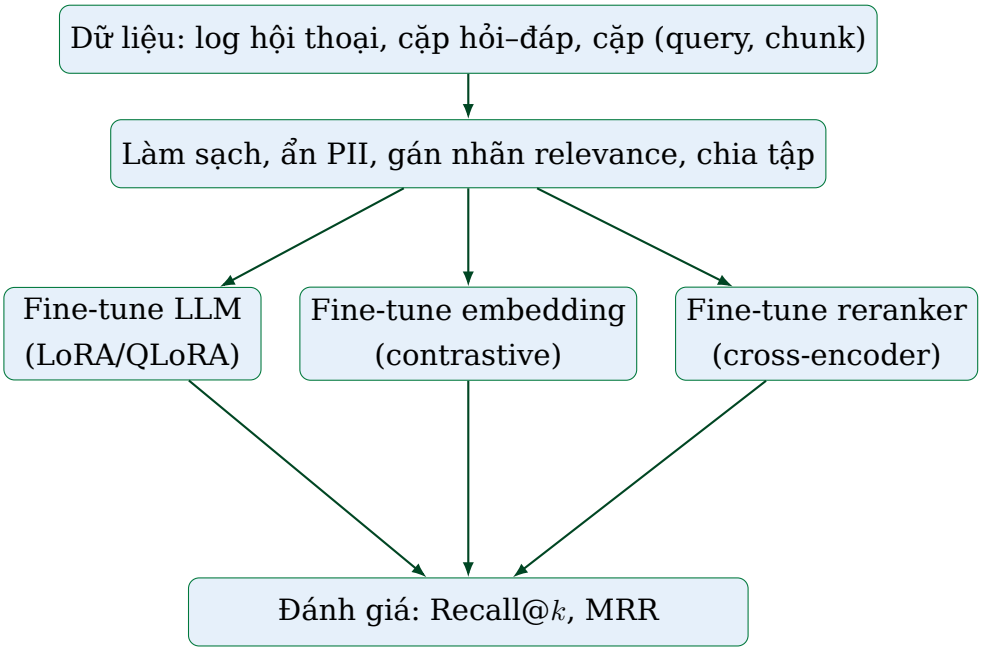
Ta dùng LoRA [14] thay vì fine-tune toàn phần để tiết kiệm bộ nhớ và dễ đóng gói adapter độc lập: gắn LoRA vào bảy ma trận chiếu của mỗi khối attention/FFN ($\{q, k, v, o, \text{gate}, \text{up}, \text{down}\}$) với hạng $r = 32$, $\alpha = 32$. Đầu ra pooling là *last-token* và vector được chuẩn hoá L2. Chỉ dẫn (instruction) chỉ áp dụng cho phía truy vấn, giữ nhất quán giữa khai thác negative, huấn luyện và đánh giá.

Sau khi thay embedding model, cần *đánh giá lại toàn bộ chỉ mục* (re-embed corpus) và đo lại Recall@k, MRR để xác nhận cải thiện trước khi đưa vào sản xuất. Cấu hình chi tiết được trình bày ở mục 4.4 và kết quả ở mục 5.3.

3.5.4 Fine-tuning reranker

Reranker là cross-encoder nhận đồng thời (truy vấn, chunk) và cho điểm mức độ liên quan. Fine-tune reranker nhằm cải thiện Recall@k sau rerank – hiện còn hạn chế ở cấu hình sản xuất – nên đây là hướng xử lý trực tiếp điểm nghẽn. Dữ liệu là các cặp (truy vấn, chunk) được gán nhãn liên quan theo đáp án tham chiếu (nhãn nhị phân hoặc theo mức), với hard negatives lấy từ top-k sai. Huấn luyện bằng hàm mất mát phân loại nhị phân (cross-entropy) hoặc xếp hạng (margin/ranking loss). Mô hình nền là Qwen3-Reranker; có thể fine-tune toàn bộ hoặc dùng LoRA. Đánh giá bằng Recall@5, MRR và nDCG.

> *Ghi chú minh bạch: với LLM và reranker, pipeline fine-tuning trong báo cáo là* thiết kế đề xuất*, chưa huấn luyện thật. Với embedding model, báo cáo đã* thí điểm huấn luyện và đánh giá thật *Qwen3-Embedding-0.6B trên một bộ dữ liệu TTHC tiếng Việt; kết quả này độc lập với hạ*



Hình 3.3: Pipeline fine-tuning đề xuất cho ba loại mô hình: LLM, embedding và reranker.

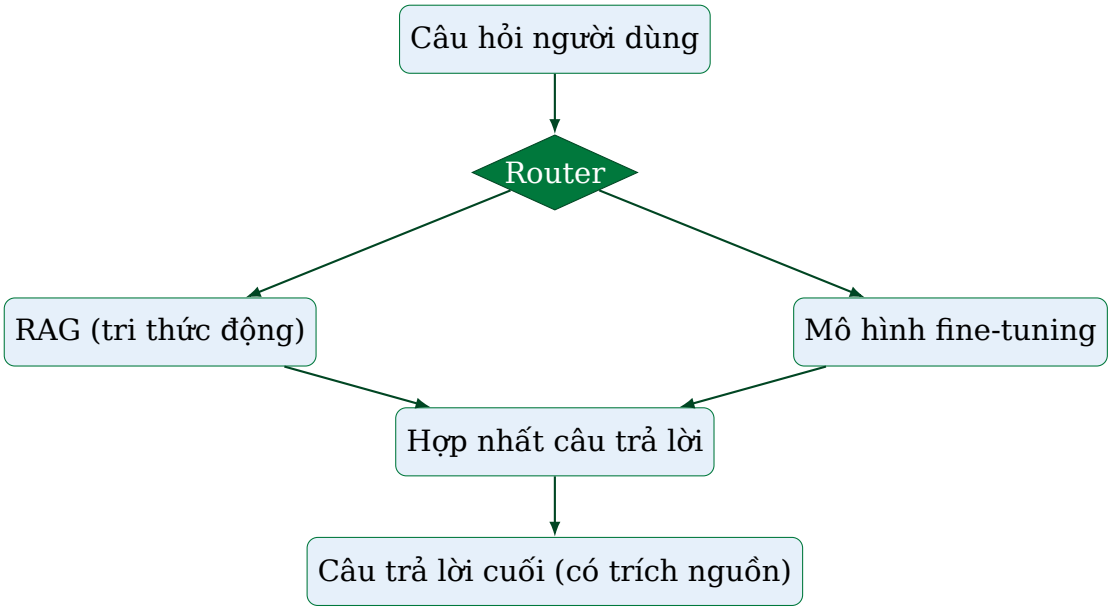
tầng và corpus sản xuất của Vietnam Post, nhằm kiểm chứng tính khả thi của pipeline.

3.6 Kiến trúc hybrid RAG + Fine-tuning

Kiến trúc hybrid sử dụng một router để quyết định chiến lược trả lời:

- **Chỉ RAG:** với câu hỏi cần tri thức động hoặc cần trích dẫn nguồn (biểu phí, quy định, số hiệu văn bản). Đây là chế độ mặc định cho phần lớn truy vấn.
- **Chỉ mô hình fine-tuning:** với câu hỏi về phong cách, định dạng, hướng dẫn quy trình ổn định, hoặc khi cần câu trả lời ngắn gọn theo mẫu nghiệp vụ.
- **Kết hợp:** mô hình fine-tuning sinh khung câu trả lời theo định dạng nghiệp vụ, sau đó RAG bổ sung bằng chứng và số liệu cụ thể; mô hình hợp nhất tạo câu trả lời cuối.

Router có thể dựa trên phân loại ý định nhẹ, hoặc để chính agent điều phối quyết định dựa trên prompt. Khi RAG không tìm thấy bằng chứng đủ tin cậy, hệ thống chuyển sang thông báo thiếu thông tin hoặc tạo ticket, tránh sinh câu trả lời không căn cứ.



Hình 3.4: Kiến trúc hybrid: router chọn RAG, mô hình fine-tuning hoặc kết hợp.

3.7 Hạ tầng và bảo mật

3.7.1 Hạ tầng phục vụ

Toàn bộ hệ thống được triển khai trên Kubernetes để bảo đảm khả năng mở rộng và ổn định. Các mô hình được phục vụ cục bộ bằng vLLM với quantization FP8. Các mô hình chính gồm: mô hình sinh ngôn ngữ (ví dụ họ Qwen với các biến thể 27B–35B), embedding model văn bản (ví dụ Qwen3-Embedding-8B), reranker (Qwen3-Reranker-8B) và mô hình OCR. Qdrant đảm nhiệm lưu trữ vector.

3.7.2 Bảo mật dữ liệu

Yêu cầu bảo mật được thực thi ở nhiều lớp: (i) mô hình chạy cục bộ, dữ liệu không rời hạ tầng; (ii) kiểm soát truy cập theo vai trò (RBAC) đối với dữ liệu nhạy cảm; (iii) ẩn PII trong log trước khi dùng cho phân tích hoặc fine-tuning; và (iv) phân quyền ở tầng dữ liệu, bảo đảm người dùng chỉ truy cập được phạm vi cho phép.

3.8 Kiến trúc quan sát: OTel Gateway → Langfuse

3.8.1 Nguyên tắc thiết kế

Hệ thống được quan sát theo nguyên tắc *trung lập backend*: agent chỉ phát vết theo chuẩn OpenTelemetry semantic conventions và *không phụ thuộc* vào backend nào. Toàn bộ việc kết nối tới backend được gom về một điểm duy nhất là **OTel Gateway** (OTel Collector). Nhờ đó, thêm một backend mới chỉ cần thêm một pipeline trong cấu hình collector, không phải sửa code agent và không cần restart agent.

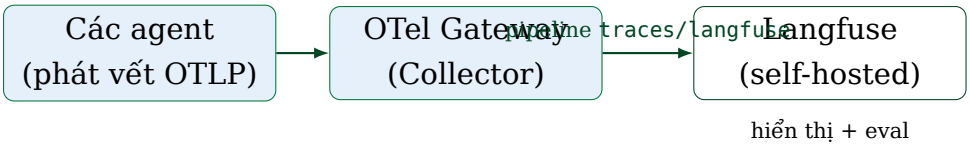
Agent phát vết qua OTLP tới collector; collector nhận ở hai giao thức gRPC (cổng 4317) và HTTP (cổng 4318), rồi chuyển tiếp tới Langfuse.

3.8.2 Pipeline gửi tới Langfuse

Pipeline `traces/langfuse` đi qua một *transform processor* ánh xạ attribute trung lập sang data model của Langfuse, rồi export qua OTLP/HTTP tới endpoint ingest của Langfuse, kèm hàng đợi gửi (sending queue) và cơ chế thử lại (retry on failure) để không mất vết khi backend chậm.

Phép ánh xạ chính thực hiện ở collector gồm:

- `service.name` làm tên trace, khớp với `OTEL_SERVICE_NAME` của agent.
- `meta.<key>` → `langfuse.trace.metadata.<key>` để dùng làm bộ lọc trong UI.
- Loại observation suy từ `gen_ai.operation.name`: `chat` → generation, `invoke_agent` → agent (hiện trên Agent Graph), `execute_tool` → tool.
- Input/output của generation và root span: `gen_ai.input.messages`, `gen_ai.output.messages`.
- Input/output của tool: `gen_ai.tool.call.arguments` và `gen_ai.tool.call.result`.
- Input/output của agent từ xa qua A2A: `input.message`, `output.message`.
- TTFT: `gen_ai.response.first_token_time` → thời điểm bắt đầu sinh token.



Hình 3.5: Kiến trúc quan sát: agent phát vết qua OTel Gateway tới Langfuse.

3.8.3 Định danh phiên và người dùng

`session.id` và `user.id` được gắn trên các span chính và truyền qua *OTel Baggage* trong carrier `otel_context` gửi kèm theo A2A/MCP. Phía nhận, một baggage processor copy hai entry này lên mọi span, kể cả span sinh ra ở process khác. Nhờ đó, một hội thoại gồm nhiều lượt và đi qua nhiều agent vẫn được gom thành một session khi xem trên Langfuse (tab Traces / Sessions / Agent Graph).

3.9 Triển khai trên Kubernetes

3.9.1 Tổ chức manifests

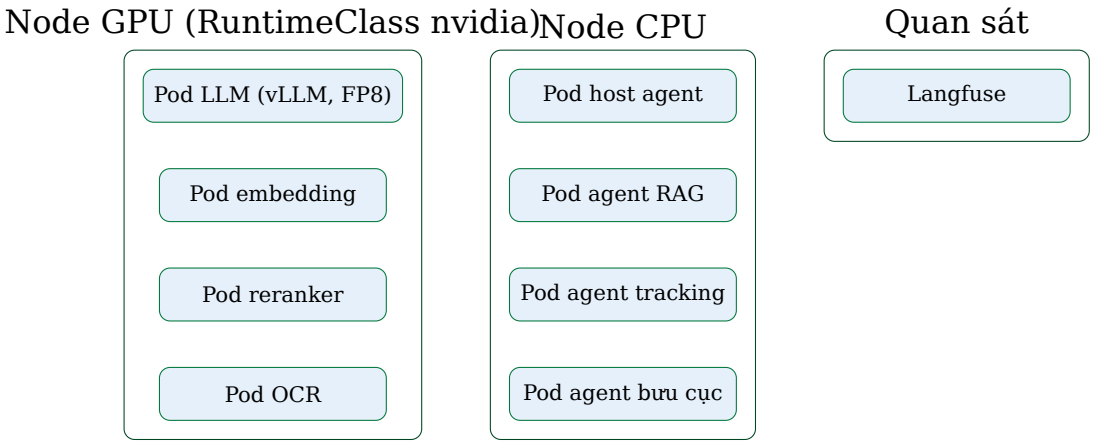
Các dịch vụ được đóng gói bằng Docker và triển khai trên Kubernetes bằng bộ manifests tổ chức theo kiểu Kustomize: mỗi dịch vụ một thư mục gồm `workloads` (Deployment), `configs` (ConfigMap), `secrets` (Secret), `networks` (Service, Ingress) và `images`. Trong giai đoạn phát triển và thử nghiệm cục bộ, hệ thống chạy bằng Docker Compose; khi đưa vào vận hành thì chuyển lên Kubernetes.

3.9.2 Workload GPU và init container

Với các workload phục vụ mô hình, Pod yêu cầu `runtimeClassName: nvidia` để dùng GPU, đồng thời dùng `nodeSelector` để neo về node có card đồ họa. Khởi động Pod có thêm *init container* phụ trách chuẩn bị dữ liệu trước khi container chính chạy:

- **Với workload mô hình:** init container tải trọng số từ máy chủ lưu trữ nội bộ (SFTP) về *PersistentVolumeClaim* (PVC), sau đó container chính khởi động vLLM và phục vụ mô hình.
- **Với workload agent:** init container tải mã agent từ kho Git nội bộ về PVC, sau đó container chính chạy API server.

Cách này giúp giữ image sát cơ bản, tách phần dữ liệu/biến động ra khỏi image, và cho phép cập nhật mô hình hoặc mã agent mà không cần build lại image.



Hình 3.6: Topologies triển khai trên Kubernetes: workload GPU, workload agent và quan sát.

3.9.3 Cấu hình, bản mật và registry

Cấu hình runtime đặt ở ConfigMap, thông tin nhạy cảm (khóa xác thực, khóa API) đặt ở Secret, riêng biến môi trường theo môi trường triển khai được quản lý cùng bộ manifests. Image lấy từ registry nội bộ của Tổng công ty; các tên miền nội bộ cần phân giải được khai báo qua `hostAliases`. `terminationGracePeriodSeconds` được đặt đủ dài để Pod dừng an toàn, không làm gián đoạn yêu cầu đang xử lý.

Kết chương

Chương này đã phân tích bài toán Vietnam Post, xác định yêu cầu và đề xuất kiến trúc multi-agent kết hợp RAG lai với fine-tuning theo mô hình hybrid, cùng hai thành phần nền tảng: kiến trúc quan sát dựa trên OTel Gateway và cách triển khai trên Kubernetes. Chương tiếp theo mô tả việc triển khai thực nghiệm các thành phần này cùng các tham số cụ thể.

Chương 4

Triển khai thực nghiệm

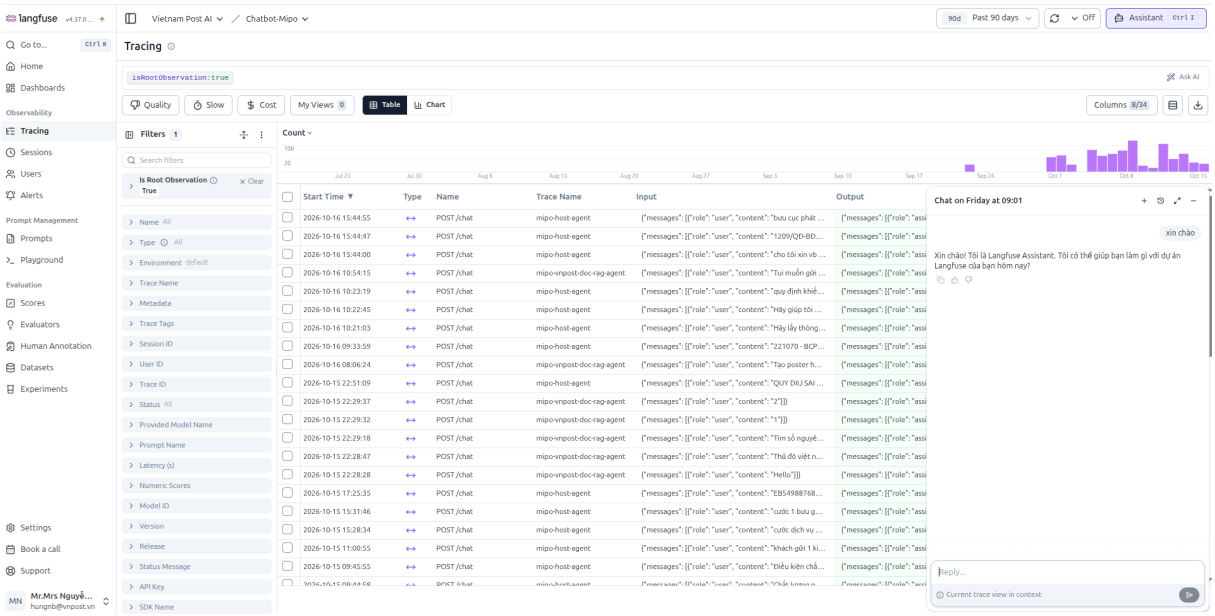
Chương này trình bày việc triển khai hệ thống theo kiến trúc đã đề xuất ở chương 3. Mục 4.1 mô tả môi trường và công cụ; mục 4.2 mô tả dữ liệu; mục 4.3 trình bày cấu hình RAG; mục 4.4 trình bày thiết kế fine-tuning; mục 4.5 mô tả triển khai multi-agent; mục 4.6 trình bày triển khai hệ thống quan sát qua OTel Gateway; mục 4.7 trình bày triển khai hệ thống lên Kubernetes; và mục 4.8 tổng kết khó khăn cùng giải pháp.

4.1 Môi trường và công cụ

Hệ thống được triển khai theo mô hình microservice trên nền Kubernetes, đóng gói bằng Docker. Bảng 4.1 liệt kê các công cụ chính.

Bảng 4.1: Các công cụ và thành phần chính của hệ thống.

Thành phần	Vai trò
vLLM	Phục vụ inference LLM, embedding, reranker và OCR với FP8
Qdrant	Vector database, lưu dense vector và sparse
Kubernetes	Điều phối container, mở rộng và ổn định dịch vụ
Docker Compose	Triển khai cục bộ phục vụ phát triển và thử nghiệm
MLflow	Quản lý phiên bản prompt và gateway
Langfuse	Thu thập và trực quan hóa inference trace
OpenTelemetry	Chuẩn thu thập trace, metric, log
Airflow	Lập lịch thu thập và cập nhật dữ liệu định kỳ
PostgreSQL	Lưu trữ dữ liệu nghiệp vụ (bưu cục, tracking)



Hình 4.1: Giao diện Langfuse và trợ lý AI tích hợp, dùng để quan sát và phân tích phiên hội thoại.

4.2 Dữ liệu

4.2.1 Các kho tài liệu

Đề tài sử dụng ba kho tài liệu đại diện cho các miền nghiệp vụ khác nhau, tóm tắt ở bảng 4.2.

Bảng 4.2: Thống kê các kho tài liệu sử dụng trong thực nghiệm.

Kho tài liệu	Số tài liệu	Nội dung tiêu biểu
Văn thư nội bộ	17	Quyết định, công văn, quy trình nghiệp vụ
Sản phẩm – dịch vụ	119	Biểu phí, quy định hàng hóa, bồi thường, thời gian toàn trình
Chuyển phát nhanh EMS	≈ 30	Biểu phí EMS, quy định hàng hóa đặc biệt, bồi thường

Kho tài liệu sản phẩm – dịch vụ gồm 114 tệp văn bản, 3 tệp PDF và 2 tệp docx; sau khi chuẩn hóa, mỗi tài liệu được sinh thêm một tệp Markdown và gắn metadata (tiêu đề, số hiệu, ngày, tóm tắt). Kho văn thư gồm các quyết định và công văn về cước phí, thời gian toàn trình, hàng hóa cấm gửi và chính sách bồi thường.

4.2.2 Bộ mẫu đánh giá retrieval

Đánh giá retrieval sử dụng một bộ mẫu nhỏ (6 câu hỏi) với đáp án tham chiếu, dùng để kiểm tra chi tiết theo từng nhánh (BM25, dense), tập ứng viên sau khi hợp và bước rerank.

4.2.3 Tiền xử lý

Quy trình tiền xử lý gồm: OCR cho bản scan; chuyển đổi sang Markdown; trích xuất metadata; structure-aware chunking; và sinh file băm (hash) để theo dõi thay đổi tài liệu. Đối với tiếng Việt, cấu hình sparse retrieval được điều chỉnh: tắt stemming và loại bỏ danh sách từ dừng, vì stemming tiếng Việt kém hiệu quả với thuật ngữ chuyên ngành và số hiệu văn bản.

4.3 Cài đặt RAG

4.3.1 Chunking và embedding

Văn bản được chia thành các chunk với độ dài tối đa khoảng 1000 ký tự và độ chồng lấn bảo toàn ngữ cảnh. Bộ chunking ưu tiên ranh giới theo tiêu đề và giữ nguyên bảng. Mỗi chunk được embedding bằng mô hình Qwen3-Embedding với vector 4096 chiều; song song đó, sparse vector cho BM25 được tạo với chỉ số IDF.

4.3.2 Cấu hình retrieval

Bảng 4.3 trình bày các tham số retrieval chính. Ở nhánh sản phẩm – dịch vụ của hệ thống multi-agent, số ứng viên BM25 là 10 và số ứng viên sau rerank là 10; ở cấu hình mẫu (template) dùng cho đánh giá retrieval, các giá trị tương ứng là 20/20/5.

4.3.3 Embedding câu hỏi và prompt

Câu hỏi được embedding với chỉ dẫn (instruction) phù hợp cho retrieval. Prompt hệ thống của agent RAG được quản lý tập trung qua MLflow Prompt Registry (ví dụ `tct-rag-agent`), cho phép cập nhật mà không cần triển khai

Bảng 4.3: Cấu hình tham số retrieval RAG.

Tham số	Cấu hình mẫu	Nhánh SPDV
Số ứng viên dense (k_1^{dense})	20	20
Số ứng viên BM25 (k_1^{bm25})	20	10
Số chunk sau rerank (k_2)	5	10
Ngưỡng điểm rerank	0,0	0,0
Số chiều embedding	4096	4096
Độ dài chunk tối đa	≈ 1000	≈ 1000

lại mã nguồn. Prompt quy định rõ: chỉ trả lời dựa trên ngữ cảnh retrieval, trích nguồn, và từ chối khi thiếu thông tin.

4.4 Thiết kế fine-tuning

Trong ba loại mô hình, pipeline cho **embedding model** đã được thí điểm triển khai và đánh giá thật trên Qwen3-Embedding-0.6B với một bộ dữ liệu TTHC tiếng Việt; **LLM** và **reranker** vẫn ở mức *đề xuất*. Mục này trình bày dữ liệu (4.4.1) và cấu hình huấn luyện (4.4.2), cách đóng gói phục vụ (4.4.3), rồi các đề xuất cho LLM và reranker.

4.4.1 Dữ liệu thí điểm cho embedding model

Bộ dữ liệu gồm 199 văn bản TTHC, chia thành 179 văn bản huấn luyện và 20 văn bản kiểm định theo tỷ lệ $0{,}1$ (hạt giống ngẫu nhiên 42 để tái lập). Sau khi khử trùng lặp theo văn bản, tập kiểm định còn 708 truy vấn duy nhất và 1.319 đoạn văn bản. Mỗi mẫu huấn luyện là bộ ba $(\text{query}, \text{positive}, \text{negative})$, với tối đa $k = 3$ hard negative cho mỗi truy vấn; tập kiểm định chia theo văn bản nên không rò rỉ chunk giữa hai tập. Bảng 4.4 tổng hợp thống kê.

4.4.2 Cấu hình huấn luyện và đánh giá

Bảng 4.5 liệt kê cấu hình huấn luyện. Mô hình được huấn luyện trên một GPU Tesla T4 trong 202,5 phút, bộ nhớ đỉnh được cấp phát 7,59 GB (52,1% dung lượng GPU). Đánh giá dùng `InformationRetrievalEvaluator` của `sentence-transformers`, với truy vấn và kho văn bản được khử trùng lặp theo văn bản (mỗi chuỗi chỉ xuất hiện một lần, một truy vấn có thể có nhiều

Bảng 4.4: Thống kê bộ dữ liệu thí điểm cho embedding model.

Tham số	**Giá trị**
Văn bản (train / val)	179 / 20
Tỷ lệ kiểm định (theo văn bản)	0,1
Truy vấn (train / val)	6.606 / 717
Bộ ba (train / val)	18.569 / 2.112
Truy vấn val duy nhất	708
Đoạn văn bản val duy nhất	1.319
Số negative tối đa mỗi truy vấn	3
Negative bị loại khi làm sạch	1.491
Bộ ba bị loại (vượt hạn mức)	990
Hạt giống	42

positive) để tránh hiện tượng "trần điểm giả" khi văn bản trùng lặp xuất hiện nhiều lần trong kho.

4.4.3 Đóng gói và phục vụ

Adapter LoRA của checkpoint tốt nhất được gộp vào mô hình nền ở độ chính xác fp32 rồi lưu fp16. Mô hình gộp được chuyển sang định dạng GGUF (F16) bằng `convert_hf_to_gguf.py` và lượng tử hoá về Q8_0 bằng `llama-quantize` của `llama.cpp` [25]; cấu hình pooling (last-token) và chuẩn hoá L2 được giữ khớp với lúc huấn luyện. Mô hình GGUF được phục vụ qua `llama-server` ở chế độ embedding với `-ngl 99` (đẩy toàn bộ layer lên GPU) và `-c 2048`, cung cấp API tương thích OpenAI tại `/v1/embeddings`. Vì mô hình không tự thêm tiền tố instruction, phía client phải thêm chuỗi tiền tố cho truy vấn (tài liệu giữ nguyên văn bản thô).

4.4.4 Đề xuất cho LLM và reranker

Với LLM và reranker, đây vẫn là phần *thiết kế đề xuất*. Bảng 4.6 trình bày cấu hình LoRA/QLoRA cho LLM và bảng 4.7 cho reranker.

Quy trình chung cho phần đề xuất: làm sạch dữ liệu $\rightarrow$ định dạng theo loại mô hình $\rightarrow$ chia tập theo câu hỏi $\rightarrow$ huấn luyện $\rightarrow$ đánh giá trên tập kiểm định $\rightarrow$ đóng gói để phục vụ. Lưu ý riêng cho embedding: sau khi thay mô hình phải *re-embed toàn bộ corpus* và dựng lại chỉ mục Qdrant trước khi đo lại Recall@k.

Bảng 4.5: Cấu hình huấn luyện và đánh giá embedding model (thí điểm).

Tham số	**Giá trị**
Mô hình nền	`unsloth/Qwen3-Embedding-0.6B`
Phương pháp	LoRA, $r = 32$, $\alpha = 32$, 7 ma trận chiếu
Độ dài chuỗi tối đa	1.024 token
Hàm mất mát	MultipleNegativesRankingLoss ($\tau^{-1} = 20$)
Bộ lấy mẫu batch	`NO_DUPLICATES`
Số epoch	2
Batch hiệu dụng	256 (64 $\times$ 4 bước tích luỹ)
Tốc độ học / scheduler	3×10^{-5} / cosine, warmup 0,05
Độ chính xác	fp16
Đánh giá / checkpoint	mỗi 20 bước
Chỉ số chọn mô hình tốt nhất	NDCG@10 trên tập kiểm định
Pooling / chuẩn hoá	last-token / L2

Bảng 4.6: Cấu hình LoRA/QLoRA đề xuất cho fine-tuning LLM.

Tham số	**Giá trị đề xuất**
Phương pháp	LoRA / QLoRA (4-bit NF4)
Hạng r	16
Hệ số α	32
Dropout	0,05
Module áp dụng	q, k, v, o, gate, up, down
Tốc độ học	1×10^{-4} đến 2×10^{-4}
Số epoch	2–3
Độ dài ngữ cảnh	4096 token
Chiến lược	SFT, có thể bổ sung DPO

Bảng 4.7: Cấu hình fine-tuning reranker đề xuất.

Tham số	**Giá trị đề xuất**
Mô hình nền	Qwen3-Reranker (cross-encoder)
Đầu vào	Cặp (truy vấn, chunk)
Nhãn	Liên quan / không liên quan theo đáp án tham chiếu
Hàm mất mát	Cross-entropy (phân loại) hoặc margin/ranking loss
Mẫu âm	Hard negatives từ top-k sai
Hạng r (nếu dùng LoRA)	16
Tốc độ học	1×10^{-5} đến 2×10^{-5}
Số epoch	2–4
Đánh giá	Recall@5, MRR, nDCG (trước/sau)

4.5 Triển khai multi-agent

4.5.1 Agent điều phối

Agent điều phối (host) tiếp nhận yêu cầu, duy trì ngữ cảnh hội thoại và quyết định định tuyến. Prompt của host quy định danh sách agent được phép gọi, tiêu chí định tuyến và cơ chế hợp nhất kết quả. Host cũng hỗ trợ tạo ticket khi cần chuyển sang hỗ trợ thủ công, với phân loại ticket theo nhóm nghiệp vụ (tư vấn, hỗ trợ ứng dụng, cước phí, đối soát thanh toán, hỗ trợ kỹ thuật).

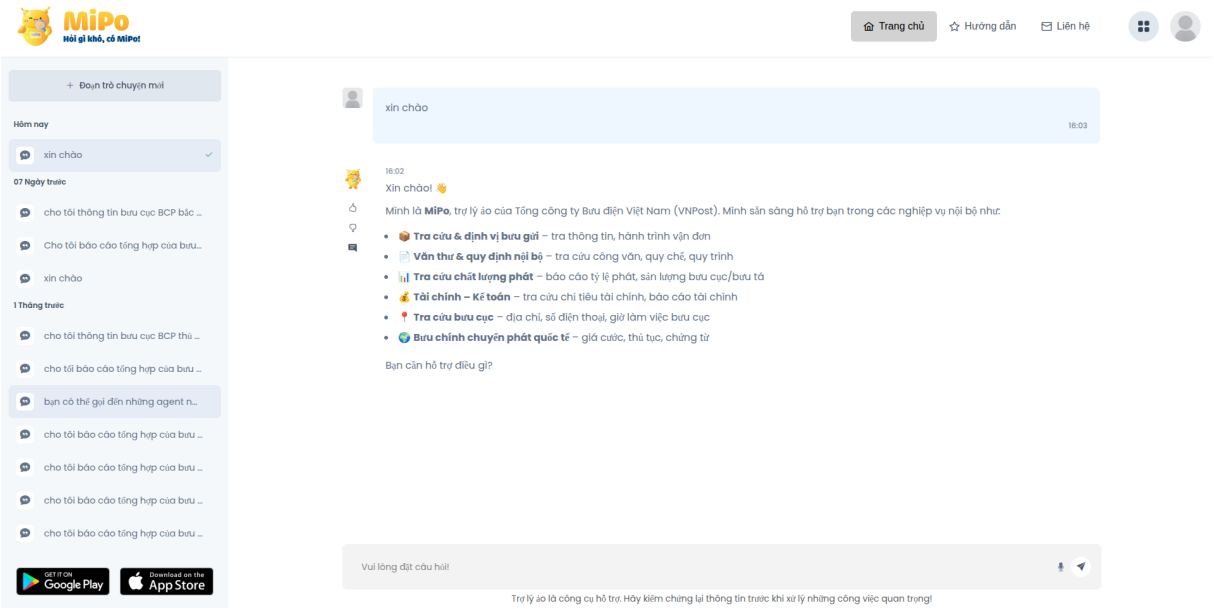
4.5.2 Các agent chuyên biệt và công cụ

Các agent chuyên biệt giao tiếp qua A2A và truy cập công cụ qua MCP. Ví dụ, agent tra cứu bưu gửi cung cấp công cụ tra cứu theo SOAP và theo API nội bộ, cùng công cụ phân tích điểm nghẽn (bottleneck). Agent tra cứu bưu cục truy vấn cơ sở dữ liệu PostgreSQL có tìm kiếm mờ (fuzzy search) để xử lý sai chính tả. Agent RAG sản phẩm – dịch vụ dùng Qdrant và reranker như mô tả.

4.5.3 Quy tắc định tuyến

Với miền EMS, hệ thống sử dụng một bộ khoảng 26 quy tắc chuyển agent, được nhóm theo các chủ đề: yêu cầu trực tiếp, khiếu nại/bồi thường, phản

hồi, tác vụ vận hành, nghiệp vụ/hợp đồng, khách hàng VIP, an ninh/pháp lý, độ tin cậy thấp và chuyển cấp. Các quy tắc được lưu dưới dạng JSON có thể đọc bằng máy, bao gồm thông tin cần thu thập và chính sách theo giờ hành chính, giúp việc định tuyến nhất quán và dễ bảo trì.



Hình 4.2: Giao diện trợ lý ảo MiPo: khung hội thoại, danh sách phiên và các nhóm nghiệp vụ được hỗ trợ.

4.6 Quan sát với OTel Gateway

4.6.1 Cấu hình OTel Gateway

OTel Gateway được cài đặt bằng image `otel/opentelemetry-collector-contrib` với một tệp cấu hình gồm ba phần: *receivers*, *processors* và *exporters*. Receiver `otlp` nhận vết qua gRPC ở cổng 4317 và HTTP ở cổng 4318; processor `batch` gom các vết trước khi gửi. Pipeline `traces/langfuse` đi qua processor `transform/langfuse` để ánh xạ attribute trung lập sang data model của Langfuse, rồi export bằng `otlphttp/langfuse` tới endpoint ingest của Langfuse.

Exporter `otlphttp` tự POST tới `{endpoint}/v1/traces` với header xác thực Basic, kèm `sending_queue` (kích thước 1000) và `retry_on_failure` (chu kỳ thử lại tăng dần từ 5 giây tới 30 giây, thời hạn 5 phút) nhằm không mất vết khi backend chậm. Các pipeline tương tự được tạo cho `metrics` và

logs.

4.6.2 Attribute agent phát

Mỗi agent phát attribute theo chuẩn OpenTelemetry semantic conventions, ở dạng trung lập mọi backend. Bảng 4.8 liệt kê các attribute chính.

Bảng 4.8: Attribute chính mà agent phát theo chuẩn OpenTelemetry.

Attribute	Ý nghĩa
`OTEL_SERVICE_NAME`	Tên service hiện trên trace
`gen_ai.operation.name`	Loại thao tác: `chat`, `invoke_agent`, `execute_tool`
`gen_ai.request.model` + `gen_ai.request.*`	Mô hình và tham số sampling
`gen_ai.usage.input/output/total_tokens`	Lượng token sử dụng
`gen_ai.input.messages` / `gen_ai.output.messages`	Nội dung messages chuẩn OpenAI
`gen_ai.tool.call.arguments` / `.result`	Đầu vào/đầu ra của tool
`gen_ai.response.first_token_time`	Thời điểm sinh token đầu tiên (TTFT)
`session.id` / `user.id`	Định danh phiên và người dùng
`meta.<key>`	Metadata của request (dùng làm bộ lọc)

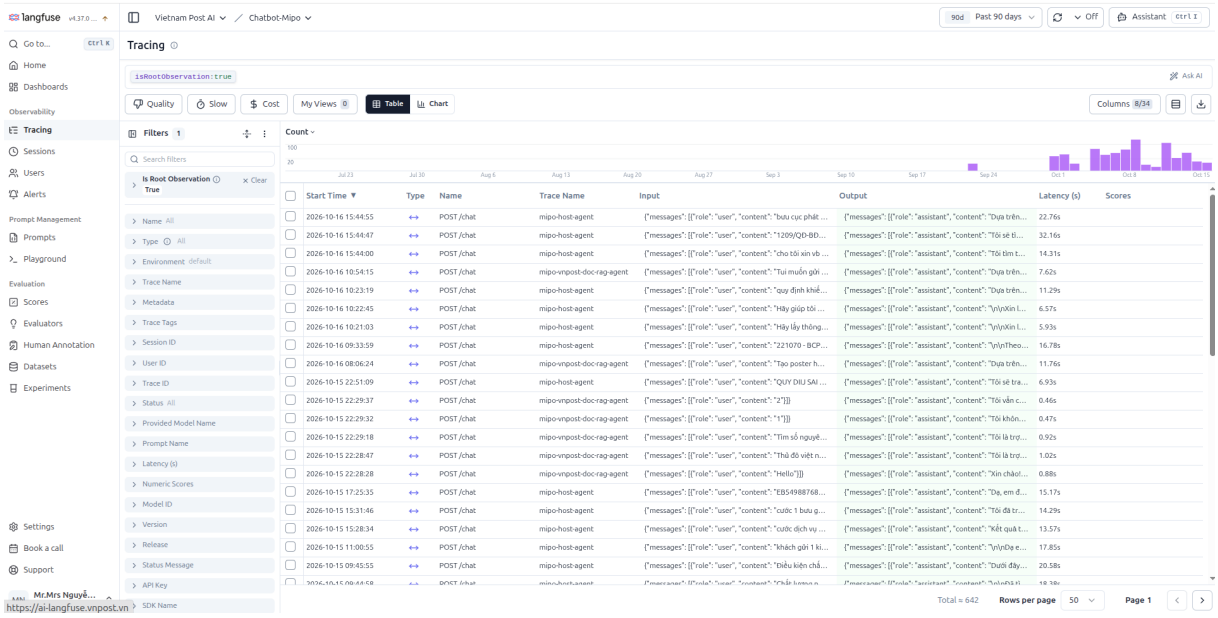
4.6.3 Ánh xạ sang Langfuse ở collector

Phép ánh xạ được khai báo ở processor `transform/langfuse` của collector, gồm: `service.name` làm tên trace; `meta.<key>` chuyển thành `langfuse.trace.metadata.<key>` để lọc trong UI; loại observation suy từ `gen_ai.operation.name`; input/output của từng observation copy từ các attribute tương ứng; và TTFT ánh xạ thành thời điểm bắt đầu sinh token. Langfuse tự nhận diện các attribute chuẩn như `session.id`, `user.id`, `gen_ai.usage.*` và `gen_ai.request.*` mà không cần ánh xạ thủ công.

Vì phép ánh xạ nằm ở collector, việc thêm hoặc đổi backend chỉ cần sửa tệp cấu hình collector và khởi động lại collector – không cần sửa code agent và không cần restart agent. Đây là điểm chính giúp kiến trúc quan sát bền vững khi hệ thống mở rộng.

4.6.4 Quy trình sử dụng

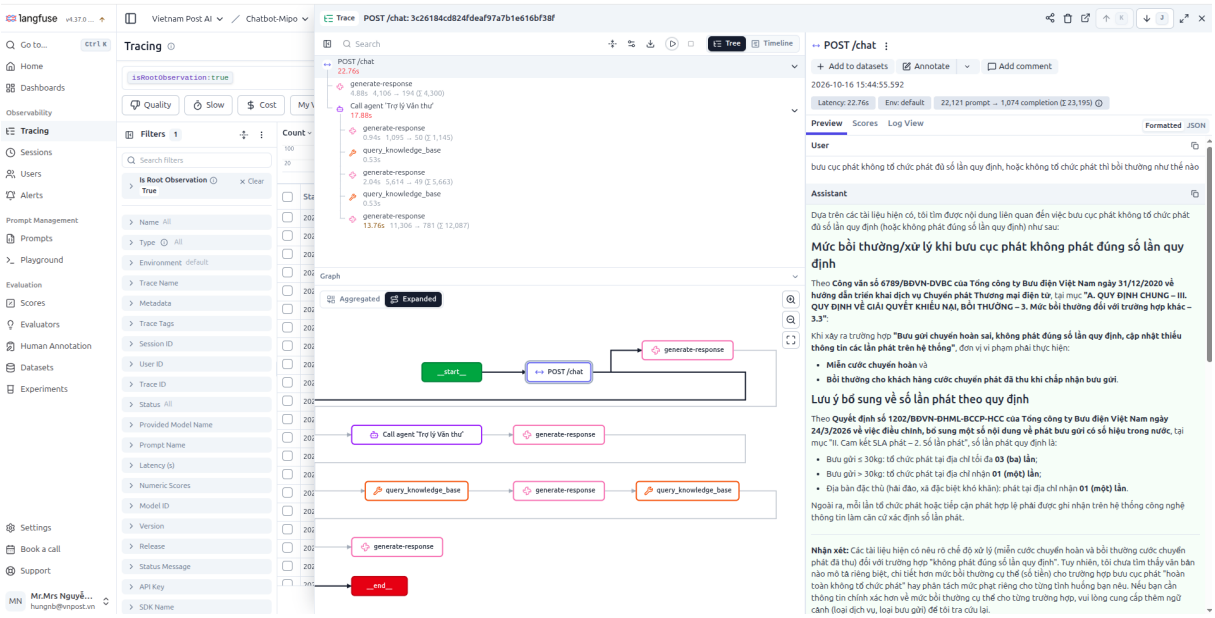
Vết được xem trên Langfuse UI ở các tab Traces, Sessions và Agent Graph. Thông tin phiên và người dùng được truyền qua OTel Baggage trong carrier `otel_context` gửi kèm theo A2A/MCP; phía nhận, baggage processor copy hai entry này lên mọi span, kể cả span sinh ra ở process khác. Nhờ đó, một hội thoại nhiều lượt đi qua nhiều agent vẫn gom thành một session.



Hình 4.3: Danh sách trace trên Langfuse (lọc `isRootObservation=true`): mỗi dòng là một lượt `/chat`, kèm agent xử lý và độ trễ.

> *Quy trình đánh giá (eval) chủ động tắt OpenTelemetry, để vết của quá trình đánh giá không lẫn vào backend quan sát và không làm nhiễu số liệu vận hành.*

Prompt được quản lý tập trung qua MLflow Prompt Registry. Mỗi agent tham chiếu một prompt theo tên và nhãn (ví dụ `@production`), nhờ đó có thể cập nhật prompt cho tất cả agent mà không cần sửa mã hay triển khai lại. Cơ chế này tách cấu hình khỏi logic và hỗ trợ kiểm soát thay đổi.



Hình 4.4: Chi tiết một trace: các bước `generate-response`, `query_knowledge_base` và lời gọi agent, kèm nội dung hội thoại và trích dẫn ở panel phải.

4.7 Triển khai hệ thống lên Kubernetes

4.7.1 Tổ chức manifests

Mỗi dịch vụ được tổ chức thành một thư mục manifests riêng gồm các nhóm tệp: `workloads` (Deployment), `configs` (ConfigMap), `secrets` (Secret), `networks` (Service, Ingress) và `images`. Cách tổ chức này giúp dễ quản lý, dễ tạo bản mới và tái sử dụng cấu hình giữa các dịch vụ. Bảng 4.9 liệt kê nhóm dịch vụ chính được triển khai.

Bảng 4.9: Nhóm dịch vụ triển khai trên Kubernetes.

Nhóm	**Thành phần**
Workload mô hình (GPU)	LLM, embedding model, reranker, OCR
Workload agent (CPU)	host agent, agent RAG, agent tracking, agent bưu cục
Quan sát	Langfuse, OTel Gateway
Dữ liệu	Qdrant, PostgreSQL, MinIO
Nền tảng	Airflow, mem0, registry nội bộ

4.7.2 Workload GPU và init container

Các Pod phục vụ mô hình yêu cầu `runtimeClassName: nvidia` để dùng GPU, và dùng `nodeSelector` để neo về node có card đồ họa. `terminationGracePeriodSeconds` đặt đủ dài để Pod dừng an toàn. Việc khởi động Pod có init container phụ trách chuẩn bị dữ liệu:

- **Workload mô hình:** init container tải trọng số từ máy chủ lưu trữ nội bộ (SFTP) về PVC; container chính sau đó khởi động vLLM và phục vụ mô hình.
- **Workload agent:** init container tải mã agent từ kho Git nội bộ về PVC; container chính sau đó chạy API server.

Cách này giữ image sát cơ bản, tách dữ liệu/biến động khỏi image, và cho phép cập nhật mô hình hoặc mã agent mà không cần build lại image.

4.7.3 Cấu hình, bản mật và kết nối nội bộ

Cấu hình runtime đặt ở ConfigMap; thông tin nhạy cảm (khóa xác thực, khóa API) đặt ở Secret; biến môi trường theo môi trường triển khai được quản lý cùng bộ manifests. Image lấy từ registry nội bộ của Tổng công ty. Các tên miền nội bộ cần phân giải được khai báo qua `hostAliases`. Trong giai đoạn phát triển và thử nghiệm cục bộ, hệ thống chạy bằng Docker Compose; khi đưa vào vận hành thì chuyển lên Kubernetes, giữ nguyên cấu hình logic của agent.

4.8 Khó khăn và giải pháp

Kết chương

Chương này đã mô tả môi trường, dữ liệu, cấu hình RAG, thiết kế fine-tuning, triển khai multi-agent và hệ thống quan sát. Chương tiếp theo trình bày kết quả thực nghiệm và đánh giá.

Bảng 4.10: Khó khăn gặp phải và giải pháp áp dụng.

Khó khăn	**Giải pháp**
Stemming tiếng Việt kém hiệu quả	Tắt stemming, loại bỏ từ dừng trong BM25
Bảng biểu bị cắt vụn khi chunking	Chunking nhận biết cấu trúc, bảo toàn bảng
Tài liệu scan chất lượng thấp	Tiền xử lý OCR bằng mô hình nhận dạng chuyên dụng
Reranker làm giảm recall top-k nhỏ	Phân tích lỗi, điều chỉnh k và ngưỡng
Chi phí GPU cho mô hình lớn	Quantization FP8, phục vụ theo lô với vLLM
Dữ liệu nhạy cảm	Chạy cục bộ, ẩn PII, RBAC theo vai trò
Prompt phân tán, khó kiểm soát	MLflow Prompt Registry tập trung

Chương 5

Kết quả và đánh giá

Chương này trình bày kết quả triển khai theo hướng định tính: mô tả những gì đã thực hiện được và các nhận định rút ra từ quan sát thực tế. Mục 5.1 trình bày phương pháp đánh giá đã thiết kế; mục 5.2 trình bày kết quả khâu retrieval; mục 5.4 so sánh các hướng tiếp cận; mục 5.5 trình bày hiệu năng hệ thống; mục 5.6 nêu case study; mục 5.7 trình bày hiệu quả vận hành; và mục 5.8 thảo luận.

> *Ghi chú minh bạch: chương này trình bày kết quả theo hướng định tính – mô tả những việc đã làm được và các nhận định rút ra từ quan sát thực tế; các chỉ số đo đạc diện rộng sẽ được công bố ở đợt đánh giá tiếp theo.*

5.1 Phương pháp đánh giá

Đề tài thiết kế đánh giá theo hai lớp:

Lớp retrieval: quy trình tái lập chính xác đường đi sản xuất – với mỗi câu hỏi, hệ thống chạy sparse retrieval BM25 và dense retrieval, lấy hợp ứng viên, rerank, rồi so sánh tập kết quả với đáp án tham chiếu. Các thước đo được sử dụng gồm Recall@k cho từng nhánh, Recall sau khi hợp, Recall@k sau rerank và MRR. Điểm nổi bật của quy trình là việc *phân tách lỗi* thành hai loại: lỗi *RECALL* (chunk đúng không được truy xuất) và lỗi *RERANK* (chunk đúng có trong ứng viên nhưng bị rerank loại khỏi top-k). Cách phân tách này giúp định vị điểm nghẽn chính xác thay vì chỉ nhìn vào một con số tổng.

Lớp hiệu năng: đo throughput và latency của vLLM ở hai chế độ – một yêu cầu đơn (single-stream) và lô lớn (batch) – nhằm xác nhận hạ tầng đáp ứng được nhu cầu phục vụ đồng thời.

5.2 Kết quả khâu retrieval

5.2.1 Đã thực hiện được

Quy trình đánh giá retrieval đã được xây dựng hoàn chỉnh: có bộ mẫu với đáp án tham chiếu, quy trình chạy đúng như cấu hình sản xuất, và xuất kết quả ra file để phân tích. Quy trình này chạy được độc lập với API, giúp kiểm tra lại mọi thay đổi cấu hình (số ứng viên mỗi nhánh, số chunk sau rerank, ngưỡng điểm) mà không ảnh hưởng hệ thống đang chạy.

5.2.2 Nhận định rút ra

Qua quan sát kết quả, một số nhận định đáng chú ý:

- **Dense retrieval mạnh ở diễn đạt khác từ khóa.** Với những câu hỏi diễn đạt khác với cách viết trong tài liệu (người dùng hỏi theo cách hiểu của mình), dense retrieval truy xuất được chunk phù hợp trong khi BM25 bỏ sót. Đây là ưu thế cốt lõi khiến dense retrieval trở thành nhánh chính.
- **BM25 mạnh với từ khóa chính xác và số hiệu văn bản.** Khi người dùng nêu đúng số hiệu quyết định, mã dịch vụ hoặc thuật ngữ hiếm, BM25 khớp chính xác hơn. Vì vậy nhánh BM25 vẫn có vai trò bổ trợ, không thể loại bỏ.
- **Hybrid retrieval đảm bảo phạm vi ứng viên.** Việc hợp hai nhánh giúp tập ứng viên chứa được chunk đúng ở cả hai loại câu hỏi, là điều kiện để bước sau có cơ hội xếp đúng.
- **Rerank là điểm cần cải thiện.** Khi số chunk sau rerank nhỏ, một phần chunk đúng bị loại khỏi ngữ cảnh. Phân tách lỗi cho thấy đây là lỗi thuộc loại *RERANK*, không phải *RECALL*: chunk đúng luôn nằm trong tập ứng viên, nhưng bước xếp hạng chưa đưa nó vào top-k. Nguyên nhân khả dĩ là reranker ưu tiên các chunk "trông giống" câu hỏi về mặt từ vựng nhưng thiếu thông tin đáp án.

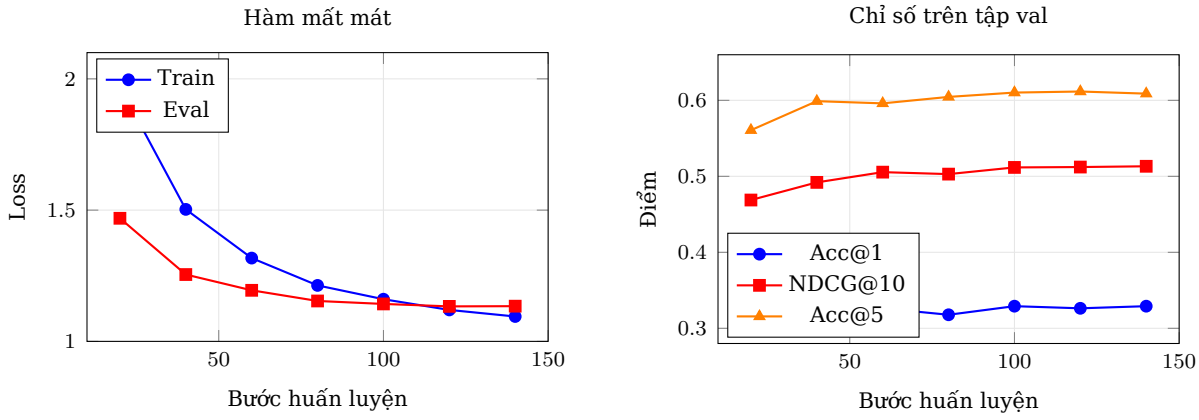
Nhận định cuối cùng có ý nghĩa thực tiễn trực tiếp: thay vì cố cải thiện retrieval, nỗ lực cải tiến nên tập trung vào reranker – ví dụ fine-tune trên dữ liệu tiếng Việt miền bưu chính, điều chỉnh số chunk sau rerank, hoặc kết hợp điểm rerank với điểm truy xuất ban đầu. Đây chính là nội dung thiết kế fine-tuning reranker ở chương 3.

5.3 Kết quả fine-tune embedding model (thí điểm)

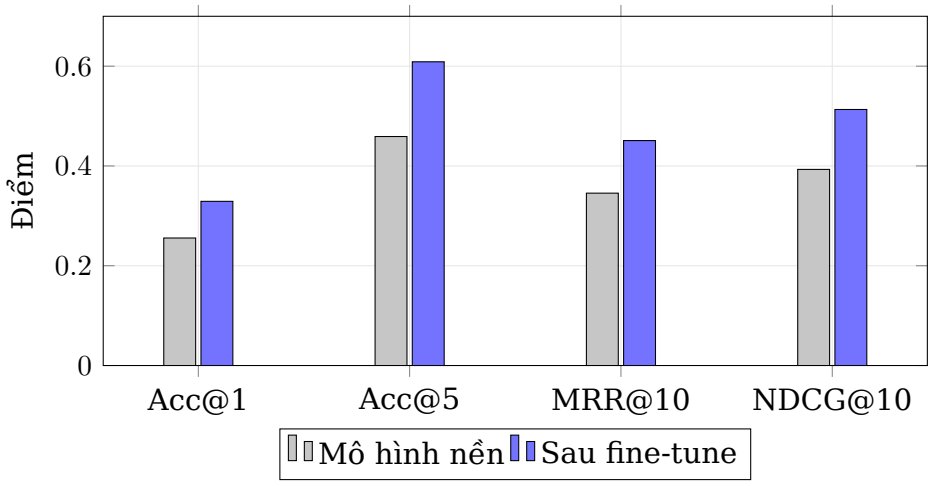
Khác với reranker và LLM còn ở mức đề xuất, embedding model đã được thí điểm huấn luyện và đánh giá thật trên bộ dữ liệu TTHC tiếng Việt (mô tả ở mục 4.4). Bảng 5.1 so sánh mô hình nền Qwen3-Embedding-0.6B và mô hình sau fine-tune trên cùng tập kiểm định (708 truy vấn / 1.319 đoạn văn bản). Mọi chỉ số đều cải thiện rõ rệt, với mức tăng tương đối từ 28,8% đến 32,6%.

Bảng 5.1: Kết quả truy xuất trên tập kiểm định TTHC (708 truy vấn / 1.319 đoạn văn bản).

Chỉ số	Mô hình nền	Sau fine-tune	Chênh lệch	Tương đối
Accuracy@1	0,2556	**0,3291**	+0,0735	+28,8%
Accuracy@3	—	0,5254	—	—
Accuracy@5	0,4590	**0,6088**	+0,1498	+32,6%
Accuracy@10	—	0,7133	—	—
MRR@10	0,3455	0,4509	+0,1054	+30,5%
NDCG@10	0,3931	**0,5132**	+0,1201	+30,6%
MAP@100	—	0,4605	—	—



Hình 5.1: Đường cong huấn luyện thí điểm. Hàm mất mát giảm đơn điệu và các chỉ số bão hoà từ khoảng bước 60 (epoch 0,8).



Hình 5.2: So sánh mô hình nền và mô hình sau fine-tune trên bốn chỉ số chính.

5.3.1 Phân tích quá trình huấn luyện

Hình 5.1 cho thấy hàm mất mát train giảm liên tục từ 1,97 xuống 1,09, còn hàm mất mát eval giảm từ 1,47 xuống 1,13 rồi chững lại ở các bước cuối; không quan sát thấy dấu hiệu overfit rõ rệt. Các chỉ số truy xuất bão hoà sớm: từ khoảng bước 60 (epoch 0,8), Accuracy@1 chỉ dao động trong khoảng 0,318–0,329 và NDCG@10 trong khoảng 0,503–0,513. Với $n = 708$ truy vấn, sai số chuẩn của Accuracy@1 xấp xỉ 0,018, nghĩa là phần lớn khác biệt giữa các bước này nằm trong nhiễu thống kê.

5.3.2 Kiểm chứng đóng gói

Bảng 5.2 trình bày kiểm chứng số học: cosine similarity giữa embedding của các artifact đóng gói (adapter, GGUF F16, GGUF Q8_0) và mô hình HF đã gộp, trên các mẫu văn bản kiểm tra. Cả ba artifact đều vượt ngưỡng an toàn, xác nhận quá trình lượng tử hoá không làm sai lệch biểu diễn.

Bảng 5.2: Kiểm chứng artifact đóng gói so với mô hình HF đã gộp.

Artifact	**Kích thước**	**Cosine nhỏ nhất**	**Ngưỡng**
Adapter vs mô hình gộp HF	80 MB	0,99965–0,99972	0,995
GGUF F16	1,2 GB	1,00006	0,99
GGUF Q8_0	610 MB	0,99846	0,99

Kết quả thí điểm cho thấy pipeline fine-tune embedding là khả thi và mang lại cải thiện rõ rệt, đồng thời xác nhận hai điều: (i) hard negative khai

thác từ mô hình nền là tín hiệu học tương phản hiệu quả; và (ii) adapter LoRA nhỏ (khoảng 80 MB) có thể đóng gói sang GGUF để phục vụ cục bộ mà gần như không mất chất lượng. Đây là cơ sở để chuyển thí điểm sang mô hình embedding lớn hơn (Qwen3-Embedding-8B) và dữ liệu sản xuất của Vietnam Post.

5.4 So sánh các hướng tiếp cận

Bảng 5.3 so sánh ba hướng tiếp cận dựa trên đặc điểm triển khai và quan sát thực tế. Với RAG, các nhận định dựa trên kết quả triển khai; với fine-tuning, báo cáo đã có kết quả thí điểm cho embedding model (mục 5.3) còn LLM và reranker vẫn mang tính phân tích thiết kế; kiến trúc hybrid là đề xuất.

Bảng 5.3: So sánh RAG, fine-tuning và kiến trúc hybrid trong bối cảnh Vietnam Post.

Tiêu chí	**RAG**	**Fine-tuning**	**Hybrid**
Tri thức động	Rất tốt	Kém	Tốt
Trích nguồn	Có	Hạn chế	Có
Phong cách miền	Trung bình	Tốt	Tốt
Chi phí cập nhật	Thấp	Cao	Thấp–trung bình
Độ trễ	Cao hơn	Thấp	Trung bình
Độ phức tạp	Trung bình	Cao	Cao
Trạng thái đánh giá	Đã triển khai	Thí điểm (embedding)	Đề xuất

Kết luận chính: RAG là lựa chọn phù hợp và khả thi nhất cho nhu cầu hiện tại của Vietnam Post vì tri thức thay đổi thường xuyên và cần trích nguồn; fine-tuning bổ trợ ở lớp hành vi; và kiến trúc hybrid là đích đến dài hạn, khi dữ liệu hội thoại đã đủ chín để huấn luyện.

5.5 Hiệu năng hệ thống

Hạ tầng phục vụ được xây dựng quanh vLLM với cơ chế PagedAttention và quantization FP8, cho phép phục vụ các mô hình cỡ 20–35 tỷ tham số trên GPU của hệ thống nội bộ. Việc đo throughput ở chế độ đơn và lô đã được thiết lập, với kết quả cho thấy mức thông lượng đủ để phục vụ đồng thời

nhu cầu của một tổng công ty, và latency chấp nhận được cho câu trả lời ngắn trong vài giây.

Điểm đáng lưu ý là quantization FP8 giảm đáng kể bộ nhớ và tăng tốc tính toán với mức suy giảm chất lượng nhỏ, giúp số lượng mô hình phục vụ song song (LLM, embedding, reranker, OCR) nằm trong khả năng của hạ tầng. Kết quả đo chi tiết các cấu hình sẽ được báo cáo khi hoàn tất chuẩn hóa quy trình benchmark.

5.6 Case study hội thoại

Để minh họa hành vi thực tế, dưới đây là một số tình huống hội thoại ví dụ (không phải bản ghi hội thoại thật).

Tình huống 1 – Hỏi biểu phí. Người dùng hỏi cước chuyển phát cho một trọng lượng và quốc gia cụ thể. Agent điều phối định tuyến đến agent RAG sản phẩm – dịch vụ; hệ thống truy xuất các chunk biểu phí liên quan, rerank, rồi sinh câu trả lời kèm trích dẫn số hiệu quyết định. Nếu không tìm thấy ngữ cảnh đủ tin cậy, hệ thống thông báo thiếu thông tin thay vì đoán.

Tình huống 2 – Tra cứu bưu gửi. Người dùng cung cấp mã bưu gửi. Agent điều phối gọi agent tra cứu bưu gửi, agent này gọi công cụ tracking qua SOAP/API và trả về tình trạng mới nhất cùng các mốc thời gian.

Tình huống 3 – Tra cứu bưu cục. Người dùng hỏi địa chỉ bưu cục gần một địa điểm. Agent tra cứu bưu cục truy vấn cơ sở dữ liệu với tìm kiếm mờ để xử lý sai chính tả và trả về danh sách bưu cục phù hợp.

Tình huống 4 – Chuyển hỗ trợ thủ công. Khi yêu cầu vượt ngoài phạm vi hoặc cần xử lý nghiệp vụ phức tạp, agent điều phối tạo ticket và chuyển đến nhân viên, bảo đảm không bỏ sót yêu cầu.

5.7 Hiệu quả vận hành

Về mặt vận hành, giải pháp mang lại hiệu quả rõ rệt ở các khía cạnh sau:

- **Giảm thời gian tra cứu:** cán bộ nghiệp vụ không phải mở nhiều văn bản để tìm điều khoản; câu trả lời kèm nguồn được trả về ngay trong hội thoại.
- **Chuẩn hóa câu trả lời:** cùng một câu hỏi nhận được câu trả lời nhất quán, theo đúng văn bản hiệu lực, giảm sai lệch giữa các điểm giao dịch.
- **Giảm tải kênh hỗ trợ thủ công:** phần câu hỏi tra cứu đơn giản được giải quyết bởi hệ thống, để nhân lực tập trung vào các yêu cầu phức tạp.
- **Hỗ trợ dữ liệu và báo cáo:** kết hợp với các agent tra cứu chất lượng, hệ thống giúp lập báo cáo và phân tích nhanh, hỗ trợ quyết định vận hành.

Ước tính định lượng về lợi ích kinh tế – xã hội (thời gian tiết kiệm, quy mô người dùng, giá trị quy đổi) sẽ được thực hiện khi có dữ liệu vận hành thực tế sau giai đoạn triển khai rộng, nhằm bảo đảm con số có căn cứ và có thể kiểm chứng.

5.8 Thảo luận

5.8.1 Ưu điểm

Hệ thống cho thấy RAG hybrid với rerank là giải pháp khả thi cho tri thức nội bộ động của Vietnam Post: dense retrieval mạnh về ngữ nghĩa, BM25 bổ trợ cho từ khóa chính xác, kiến trúc multi-agent giúp mở rộng theo nghiệp vụ, và việc phục vụ cục bộ đáp ứng yêu cầu bảo mật. Quan sát tập trung và quản lý prompt giúp vận hành và cải tiến liên tục.

5.8.2 Hạn chế

Thứ nhất, khâu rerank còn hạn chế và làm mất một phần chunk đúng ở top-k nhỏ. Thứ hai, phần fine-tuning chưa được huấn luyện thật nên chưa có bằng chứng định lượng. Thứ ba, các kết quả chủ yếu tập trung vào khâu retrieval và thí điểm embedding, chưa có đánh giá diện rộng với người dùng cuối.

5.8.3 Tính khái quát

Kiến trúc và quy trình đánh giá đề xuất có thể áp dụng cho các tổ chức có đặc thù tương tự: khối tài liệu nội bộ lớn, nhu cầu tra cứu thường xuyên, và yêu cầu bảo mật dữ liệu. Việc thay thế kho tài liệu và agent chuyên biệt cho phép tái sử dụng nền tảng cho miền khác.

Kết chương

Chương này đã trình bày những việc đã làm được ở khâu retrieval, hiệu năng và case study, cùng các nhận định định tính. Chương tiếp theo kết luận và đề xuất hướng phát triển.

Chương 6

Kết luận và hướng phát triển

6.1 Kết luận

Báo cáo đã nghiên cứu và triển khai thực nghiệm một trợ lý ảo thông minh cho Tổng công ty Bưu điện Việt Nam, dựa trên mô hình ngôn ngữ lớn kết hợp kỹ thuật RAG (RAG) và fine-tuning (fine-tuning). Các kết quả chính bao gồm:

1. **Hệ thống hóa cơ sở lý thuyết:** làm rõ kiến trúc Transformer, cơ chế của RAG (chunking, embedding, hybrid retrieval, rerank), các phương pháp fine-tuning (SFT, LoRA/QLoRA, DPO), các thước đo đánh giá và nền tảng triển khai (vLLM, FP8, MCP, A2A, OpenTelemetry, MLflow). Từ đó, báo cáo lập luận cho kiến trúc hybrid RAG + fine-tuning.
2. **Thiết kế kiến trúc multi-agent:** đề xuất kiến trúc gồm một agent điều phối và các agent chuyên biệt giao tiếp qua A2A, truy cập công cụ qua MCP, phục vụ cục bộ nhằm bảo đảm dữ liệu nội bộ không rời hạ tầng tổ chức.
3. **Triển khai pipeline RAG lai:** cài đặt sparse retrieval BM25 và dense retrieval trên Qdrant, kết hợp rerank bằng cross-encoder, cùng quy trình tiền xử lý tài liệu (OCR, Markdown hóa, metadata hóa, structure-aware chunking).
4. **Pipeline fine-tuning:** đề xuất quy trình thu thập dữ liệu, ẩn PII và fine-tuning cho cả ba loại mô hình – LLM (LoRA/QLoRA), embedding model (contrastive learning) và reranker cross-encoder – và *thí điểm* huấn luyện, đóng gói cho embedding model; kèm tiêu chí quyết định khi nào dùng RAG, fine-tuning hoặc kết hợp. Thí điểm embedding đạt Accuracy@1 tăng từ 0,2556 lên 0,3291 và NDCG@10 từ 0,3931 lên 0,5132 (tăng tương đối khoảng 29–31%).
5. **Đánh giá retrieval:** đã xây dựng quy trình đánh giá retrieval tái lập đúng đường đi sản xuất, kèm cơ chế phân tách lỗi giữa khâu retrieval và khâu rerank.

6. **Hiệu năng và hiệu quả:** hạ tầng vLLM với quantization FP8 phục vụ được các mô hình cỡ 20–35 tỷ tham số, đủ đáp ứng nhu cầu đồng thời của một tổng công ty; giải pháp giúp giảm thời gian tra cứu và chuẩn hóa câu trả lời ở khâu vận hành.

Tổng hợp lại, RAG là giải pháp phù hợp và khả thi cho nhu cầu tri thức động và trích nguồn của Vietnam Post, fine-tuning bổ trợ ở lớp hành vi và phong cách nghiệp vụ, và kiến trúc hybrid là đích đến dài hạn mang lại giá trị thực tiễn cao.

6.2 Hạn chế

Bên cạnh kết quả đạt được, đề tài còn một số hạn chế:

- **Chưa có đánh giá diện rộng với người dùng cuối:** kết quả hiện tại tập trung vào khâu retrieval và thí điểm embedding, chưa đo lường mức độ hữu ích trên người dùng thật.
- **Suy giảm recall ở khâu rerank:** cấu hình hiện tại làm mất một phần chunk đúng ở top-k nhỏ, cần cải tiến.
- **Phần fine-tuning chưa hoàn chỉnh:** embedding model đã được thí điểm và có kết quả định lượng, nhưng trên mô hình 0.6B và corpus TTHC độc lập (chưa phải dữ liệu sản xuất của Vietnam Post); LLM và reranker mới dừng ở thiết kế, chưa có bằng chứng huấn luyện thật.
- **Phạm vi miền hẹp:** thực nghiệm tập trung vào bưu chính/EMS; tính khái quát sang các miền khác chưa được kiểm chứng.

6.3 Hướng phát triển

Trên cơ sở kết quả và hạn chế, đề tài đề xuất các hướng phát triển sau:

1. **Mở rộng và chuẩn hóa đánh giá:** chuẩn hóa quy trình đo retrieval trên bộ mẫu lớn hơn (hàng trăm tới hàng nghìn câu); bổ sung đánh giá A/B với người dùng cuối.
2. **Cải tiến retrieval và rerank:** điều chỉnh k_2 và ngưỡng điểm; thử nghiệm kết hợp điểm rerank với điểm retrieval; mở rộng thí điểm fine-tune embedding lên mô hình lớn hơn (Qwen3-Embedding-8B) trên dữ liệu tiếng Việt miền bưu chính, và fine-tune reranker để nâng Recall@5 sau rerank.

3. **Nâng cấp lên agentic RAG:** cho phép agent tự quyết định truy xuất nhiều vòng, cải viết truy vấn và gọi công cụ bổ trợ khi ngữ cảnh chưa đủ.
4. **Khai thác GraphRAG:** xây dựng đồ thị tri thức từ tài liệu để truy xuất theo quan hệ giữa các thực thể (dịch vụ, biểu phí, quy định, địa bàn), hỗ trợ câu hỏi phức tạp.
5. **Triển khai fine-tuning định kỳ:** khi đã tích lũy đủ log hội thoại chất lượng, tiến hành SFT/LoRA và đánh giá so với RAG thuần; xây dựng quy trình cập nhật định kỳ.
6. **Continual learning và cá nhân hóa:** cập nhật tri thức liên tục và học sở thích người dùng, đồng thời bảo đảm kiểm soát chất lượng và an toàn.
7. **Mở rộng đa ngôn ngữ và đa kênh:** hỗ trợ thêm ngôn ngữ và tích hợp vào các kênh như ứng dụng di động, website, tổng đài.
8. **Tăng cường an toàn và tuân thủ:** kiểm soát nội dung đầu ra, chống rò rỉ dữ liệu nhạy cảm, và bảo đảm tuân thủ quy định về dữ liệu cá nhân.

Kết chương

Chương này đã tóm tắt kết quả và đóng góp của đề tài, chỉ rõ các hạn chế cần khắc phục và đề xuất lộ trình phát triển. Những kết quả này là cơ sở để Vietnam Post tiếp tục mở rộng hệ thống trợ lý ảo trong tương lai.

Phụ lục A

Prompt hệ thống mẫu

Phụ lục này trình bày các prompt hệ thống mẫu (đã lược giản) dùng cho agent điều phối và agent RAG. Prompt thực tế được quản lý tập trung qua MLflow Prompt Registry.

A.1. Prompt agent điều phối (host)

```
Bạn là trợ lý ảo của Tổng công ty Bưu điện Việt Nam (Vietnam Post).
Nhiệm vụ: hiểu yêu câu người dùng, duy trì ngữ cảnh hội thoại, và định
tuyến đến agent chuyên biệt phù hợp.

Các agent được phép gọi:
- "Trợ lý tra cứu sản phẩm dịch vụ": câu hỏi về biểu phí, quy định,
  chính sách bồi thường, thời gian toàn trình.
- "Trợ lý định vị Bưu gửi": tra cứu tình trạng bưu gửi theo mã.
- "Trợ lý tra cứu bưu cục": tra cứu thông tin/địa chỉ bưu cục.

Nguyên tắć:
1. Chỉ trả lời dựa trên thông tin từ agent chuyên biệt hoặc tài liệu.
2. Nêú thiếú thông tin, thông báo rõ và đề̀ nghị tạo ticket hỗ̃ trợ.
3. Không bịa số liệu, số hiệu văn bản hoặc chính sách.
4. Khi nhận kết quả từ agent chuyên biệt, hợp nhất thành câu trả lời
   mạch lạc, có trích dẫn nguồn khi có.
```

A.2. Prompt agent RAG

```
Bạn là agent hỏi đáp trên tài liệu nội bộ của Vietnam Post.

Bạn được cung cấp:
- Câu hỏi của người dùng.
- Các chunk ngữ cảnh truy xuất (kèm tiêu đề̀, số hiệu, đường dẫn tệp).
- Lịch sử hội thoại gần đây.

Nguyên tắć trả lời:
1. Chỉ sử dụng thông tin trong ngữ cảnh retrieval. Không dùng kiến thức
   bên ngoài để suy diễn các con số nghiệp vụ.
2. Nêú ngữ cảnh không đủ để trả lời, trả lời đúng mẫu:
   "Tôi chưa tìm thấy thông tin này trong tài liệu nội bộ."
```

```
3. Trích dẫn nguồn (tiêu đề/số hiệu văn bản) cho mỗi thông tin quan
trọng.
4. Trình bày ngắn gọn, đúng trọng tâm, dùng định dạng phù hợp (bảng nếu
   là biểu phí).
```

Phụ lục B

Bảng cấu hình tham số

Bảng B.1: Cấu hình tham số tổng hợp của hệ thống.

Tham số	**Giá trị**
Số chiều embedding	4096
Độ dài chunk tối đa	≈ 1000 ký tự
Độ chồng lấn chunk	bảo toàn ngữ cảnh liền kề
Số ứng viên dense (k_1^{dense})	20
Số ứng viên BM25 (k_1^{bm25})	10–20
Số chunk sau rerank (k_2)	5–10
Ngưỡng điểm rerank	0,0
Stemming tiếng Việt	tắt
Danh sách từ dừng	loại bỏ
Nhiệt độ sinh (temperature)	0,7
Top-p	0,80
Top-k (sampling)	20

Phụ lục C

Danh mục mô hình và thành phần

Bảng C.1: Các mô hình và thành phần chính được sử dụng.

Thành phần	Mô hình / công nghệ
Mô hình sinh ngôn ngữ	`vietnampost-llm` (gateway); họ Qwen 27B–35B (FP8)
Mô hình sinh thay thế	GPT-OSS-20B (FP8)
Mô hình embedding	Qwen3-Embedding-8B (4096 chiều)
Mô hình rerank	Qwen3-Reranker-8B (cross-encoder)
Mô hình OCR	olmocr-2-7b / lightonocr2-1b
Vector database	Qdrant v1.16.1 (dense COSINE + sparse IDF)
Cơ sở dữ liệu nghiệp vụ	PostgreSQL
Phục vụ inference	vLLM với PagedAttention và FP8
Quản lý prompt	MLflow Prompt Registry (`@production`)
Quan sát	OpenTelemetry, Langfuse
Điều phối	Kubernetes, Docker Compose, Airflow

Phụ lục D

Mẫu hội thoại minh họa

```
[Người dùng] Cước EMS đi Nhật cho bưu gửi 500g là bao nhiêu?

[Agent RAG] Theo biểu phí EMS hiện hành (trích Quyết định số ...), cước
cho bưu gửi 500g đi Nhật Bản thuộc bậc ... với mức cước ... VND.
Mức cước có thể thay đổi theo thời điểm và chính sách ưu đãi; vui lòng
kiểm tra lại trước khi gửi.

[Người dùng] Bưu gửi của tôi mã EG123456789VN đang ở đâu?

[Agent định vị] Bưu gửi EG123456789VN đã đến bưu cục phát ... vào
... . Trạng thái hiện tại: đang phát.

[Người dùng] Gần 12 Nguyễn Trãi, Hà Nội có bưu cục nào?

[Agent tra cứu bưu cục] Các bưu cục gần khu vực ... :
1. Bưu cục ... - địa chỉ ... - giờ mở cửa ...
2. Bưu cục ... - địa chỉ ... - giờ mở cửa ...
```

Các ví dụ trong phụ lục đã được lược giản và thay thế số liệu bằng dấu "..." để tránh công bố thông tin nội bộ. Mẫu hội thoại nhằm minh họa luồng định tuyến và định dạng câu trả lời, không phải dữ liệu thật.

Tài liệu tham khảo

[1] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, 2017.

[2] Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural machine translation of rare words with subword units. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 1715–1725, 2016.

[3] Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Xiaolei Wang, Yupeng Hou, Yingqian Min, Beichen Zhang, Junjie Zhang, Zican Dong, et al. A survey of large language models. *arXiv preprint arXiv:2303.18223*, 2023.

[4] Hyung Won Chung, Le Hou, Shayne Longpre, Barret Zoph, Yi Tay, William Fedus, et al. Scaling instruction-finetuned language models. *arXiv preprint arXiv:2210.11416*, 2022.

[5] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. *Advances in Neural Information Processing Systems (NeurIPS)*, 35:27730–27744, 2022.

[6] Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D Manning, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. *Advances in Neural Information Processing Systems (NeurIPS)*, 36, 2023.

[7] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems (NeurIPS)*, 35:24824–24837, 2022.

[8] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pages 9459–9474, 2020.

[9] Yu A Malkov and Dmitry A Yashunin. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 42(4):824–836, 2020.

[10] Qdrant. Qdrant vector database documentation. `https://qdrant.tech/documentation/`, 2024. Truy cập 2026.

[11] Stephen Robertson and Hugo Zaragoza. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009.

[12] Vladimir Karpukhin, Barlas Oğuz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. Dense passage retrieval for open-domain question answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 6769–6781, 2020.

[13] Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yi Dai, Jiawei Sun, Meng Wang, and Haofen Wang. Retrieval-augmented generation for large language models: A survey. *arXiv preprint arXiv:2312.10997*, 2023.

[14] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*, 2021.

[15] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. QLoRA: Efficient finetuning of quantized LLMs. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, 2023.

[16] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with

PagedAttention. In *Proceedings of the 29th ACM Symposium on Operating Systems Principles (SOSP)*, pages 611–626, 2023.

[17] vLLM Project. vllm: Easy, fast, and cheap llm serving for everyone. https://docs.vllm.ai, 2024. Truy cập 2026.

[18] Anthropic. Model context protocol specification. https://modelcontextprotocol.io, 2024. Truy cập 2026.

[19] Google. Agent2agent (a2a) protocol. https://a2a-protocol.org, 2024. Truy cập 2026.

[20] OpenTelemetry. Opentelemetry documentation. https://opentelemetry.io, 2024. Truy cập 2026.

[21] Langfuse. Langfuse: Open source llm engineering platform. https://langfuse.com, 2024. Truy cập 2026.

[22] MLflow. Mlflow: A platform for the machine learning lifecycle. https://mlflow.org, 2024. Truy cập 2026.

[23] Yanzhao Zhang, Mingxin Li, Dingkun Long, Xin Zhang, Huan Lin, Jian Yang, Pengjun Xie, An Yang, Dayiheng Liu, Junyang Lin, and Jingren Zhou. Qwen3 embedding: Advancing text embedding and ranking models. *arXiv preprint arXiv:2506.05176*, 2025.

[24] Matthew Henderson, Rami Al-Rfou, Brian Strope, Yun-hsuan Sung, László Lukács, Ruiqi Guo, Sanjiv Kumar, Balint Miklós, and Ray Kurzweil. Efficient natural language response suggestion for smart reply. *arXiv preprint arXiv:1705.00652*, 2017.

[25] Georgi Gerganov and contributors. llama.cpp: LLM inference in pure C/C++. https://github.com/ggml-org/llama.cpp, 2024.